From Classical to Quantized Deep Learning: Anomaly Detection in IoT Networks Using Logistic Regression and Quantized Autoencoders

UC Irvine/Statistical Methods: Joe Nguyen

Table of contents

1 Abstract

The rapid growth of Internet of Things (IoT) devices has significantly expanded the attack surface of modern networks, creating new challenges for intrusion detection systems. IoT environments often consist of heterogeneous devices with diverse communication patterns and limited computational resources, making it difficult to deploy traditional security mechanisms. This study investigates both supervised and unsupervised machine learning approaches for detecting anomalous network traffic within the **RT-IoT 2022 dataset**, with a focus on developing models that are both accurate and suitable for potential edge deployment.

The analysis begins with **exploratory data analysis (EDA)** to understand the structure and characteristics of the dataset. The results reveal strong class imbalance between normal and attack traffic, substantial multicollinearity among network flow features, and clear separability between many attack and normal traffic patterns. These properties suggest that both classical classification models and anomaly detection techniques may perform effectively in this setting.

We first establish **logistic regression models** as supervised baselines. Three variants were evaluated: a model trained using all features, a LASSO-based feature selection model, and a stepwise-selected feature model. Logistic regression provides an interpretable and computationally lightweight approach while maintaining strong predictive performance, demonstrating that even simple models can effectively detect malicious traffic when the feature space contains informative signals.

Next, we investigate **autoencoder-based anomaly detection**, an unsupervised approach that learns the structure of normal network traffic without requiring labeled attack data. The baseline autoencoder is trained exclusively on normal traffic and uses reconstruction error as an anomaly score. A threshold selected using validation-set F1 maximization converts reconstruction errors into attack predictions. The model achieves strong detection performance, indicating that deviations from learned normal traffic patterns provide a reliable signal for identifying anomalies.

To evaluate deployment-friendly architectures, we further explore **quantized autoencoder variants**, including a half-precision model (QAE-f16) and an 8-bit dynamically quantized model (QAE-u8). These models significantly reduce numerical precision and memory requirements compared with the standard 32-bit baseline. Despite this reduction in precision, both quantized models maintain nearly identical detection performance. The QAE-u8 model introduces a shifted reconstruction error scale requiring a different detection threshold, but classification metrics remain largely unchanged. These findings suggest that quantized autoencoders can retain strong anomaly detection capability while offering substantial efficiency gains.

While the results demonstrate promising performance, several limitations should be considered. The dataset lacks certain documented traffic classes, including Amazon Alexa traffic, which reduces the diversity of benign behaviors available for training. Additionally, strong feature correlations and clear class separation within the dataset may simplify the detection task relative to real-world IoT environments.

Overall, this study shows that both supervised logistic regression and unsupervised autoencoder models can effectively detect anomalous IoT network traffic. Moreover, quantization techniques enable neural models to achieve comparable detection performance while significantly reducing model precision requirements, highlighting their potential for **lightweight intrusion detection in resource-constrained IoT and edge computing environments**.

2 Introduction

The rapid adoption of Internet of Things (IoT) technologies across sectors such as healthcare, manufacturing, agriculture, and smart infrastructure has increased reliance on continuous connectivity and large-scale data exchange. While this connectivity enables new capabilities, it also expands the attack surface, making IoT devices attractive entry points for cyberattacks. Compromised devices can be leveraged to infiltrate broader network infrastructures through vulnerable endpoints (Sharmila and Nagapadma, 2023). The financial impact of these attacks continues to grow. The IBM Cost of a Data Breach Report 2025 estimates the **global average cost of a breach at USD 4.44 million**, highlighting the sustained economic risk posed by increasingly sophisticated and AI-enabled threats (IBM Security, 2025). Real-world incidents illustrate these risks clearly. In 2021, the Verkada breach exposed live feeds from approximately 150 million surveillance cameras, and in a separate incident, an attacker manipulated chemical levels at a Florida water treatment facility. Together, these events demonstrate the real-world consequences of cyberattacks targeting IoT systems and reinforce the need for reliable anomaly detection mechanisms as IoT deployments continue to scale (Sharmila and Nagapadma, 2023).

This project extends the work of Sharmila and Nagapadma (2023) by evaluating machine learning approaches for identifying malicious IoT network traffic within the RT-IoT 2022 dataset. In addition to model development, the study places strong emphasis on **exploratory data analysis (EDA)** to understand the statistical structure, feature relationships, and traffic patterns present in the dataset. Careful examination of feature distributions, correlations, and dimensional structure provides critical context for model design and helps explain why certain approaches perform well. Rather than focusing on operational deployment, the analysis prioritizes understanding how IoT network telemetry behaves and how machine learning models capture these patterns.

Modern intrusion detection systems (IDS) for IoT environments increasingly rely on machine learning techniques, particularly anomaly detection, to identify deviations from normal network behavior. As discussed by Sharmila and Nagapadma (2023), unsupervised approaches are especially well suited to IoT settings where labeled attack data may be incomplete and where new attack patterns may emerge over time. However, progress in this area has often been constrained by the limited availability of realistic IoT traffic datasets. The RT-IoT 2022 dataset addresses this limitation by providing network traffic generated from real IoT devices under both normal and attack scenarios, enabling systematic evaluation of anomaly detection methods.

This study therefore investigates both **supervised and unsupervised machine learning approaches** for detecting anomalous network traffic. Logistic regression models provide interpretable supervised baselines, while autoencoder-based models learn compact representations of normal network behavior and detect anomalies through reconstruction error. By combining rigorous exploratory analysis with multiple modeling strategies, the project aims to better understand how IoT network features relate to malicious activity and how different modeling approaches capture these relationships.

2.1 Project Objectives

The goal of this project is to evaluate machine learning approaches for detecting anomalous network behavior in IoT environments using the RT-IoT 2022 dataset, with particular emphasis on **data exploration, model interpretability, and comparative performance analysis**.

The project is guided by the following objectives:

2.1.1 Conduct Comprehensive Exploratory Data Analysis

- Perform systematic exploratory analysis of the RT-IoT dataset to understand feature distributions, correlations, class structure, and dimensional characteristics.
- Identify potential issues such as missing traffic categories, multicollinearity among network telemetry features, and class imbalance.
- Apply dimensionality reduction techniques such as principal component analysis (PCA) and unsupervised clustering to examine the latent structure of the feature space and evaluate whether normal and attack traffic form distinct clusters
- Provide context for interpreting model performance and guide the design of downstream machine learning models.

2.1.2 Establish a Supervised Baseline

- Develop logistic regression classifiers as interpretable baselines for distinguishing between normal and attack network traffic.
- Evaluate models using the full feature set as well as feature-reduced variants using LASSO and stepwise selection.
- These models provide a statistical reference point for understanding how effectively linear decision boundaries separate benign and malicious IoT activity.

2.1.3 Develop Autoencoder-Based Anomaly Detection Models

- Implement autoencoder architectures trained primarily on normal network traffic, using reconstruction error to identify anomalous behavior.
- Evaluate both full-precision and quantized variants (QAE-f16 & QAE-u8) of the autoencoder models.
- These models aim to learn compact representations of normal device activity and flag deviations that may correspond to cyberattacks.

2.1.4 Compare Supervised and Unsupervised Detection Approaches

- Systematically compare supervised logistic regression models with unsupervised autoencoder-based anomaly detection models.
- Evaluate performance using metrics such as accuracy, precision, recall, F1 score, ROC-AUC, and PR-AUC.
- Analyze how model assumptions, feature structure, and dataset characteristics influence detection performance.

2.2 Related Work

Recent studies emphasize the use of unsupervised learning methods, particularly autoencoders, for anomaly detection in IoT networks. Autoencoders learn the distribution of normal network behavior and detect attacks by measuring reconstruction error when anomalous traffic deviates from learned patterns. This approach has proven effective for identifying various IoT attacks such as DDoS, scanning, and spoofing attacks (Sharmila & Nagapadma, 2023; Tejaswini et al., 2025).

Other studies have focused on improving the decision threshold selection used in anomaly detection systems. Because anomaly detection typically relies on reconstruction error to distinguish between normal and malicious traffic, selecting an optimal threshold is critical for balancing false positives and false negatives. Putrada and Ilhami (2024) proposed using the **geometric mean (g-mean) metric to determine the optimal threshold** for autoencoder-based anomaly detection on the RT-IoT2022 dataset. Their results demonstrated improved accuracy and sensitivity compared with alternative thresholding approaches such as statistical significance levels, z-score methods, and Bayesian models.

In addition to deep learning approaches, traditional machine learning models continue to serve as strong baselines for intrusion detection. For example, Chythanya et al. (2025) evaluated logistic regression on the RT-IoT2022 dataset and **addressed dataset imbalance using the Synthetic Minority Oversampling Technique (SMOTE)**. Their results achieved high classification performance, with precision, recall, and F1 scores approaching 1.0 for several attack classes and an overall test accuracy of approximately 98%. However, certain classes remained difficult to classify due to overlapping network traffic characteristics.

Beyond specific algorithms, research has also explored hybrid and deep neural network architectures that combine multiple neural network models to capture spatial and temporal patterns in network traffic. For instance, Patel et al. (2024) proposed a quantized deep neural network intrusion detection system combining **convolutional neural networks (CNNs) and long short-term memory (LSTM) networks**. Their approach demonstrated strong performance for multi-class attack detection while leveraging quantization techniques to reduce computational cost.

Recent research has also explored the use of **large transformer models (LTMs)** for intrusion detection in IoT environments. Almadhor et al. (2025) investigated transformer-based architectures such as BERT, DistilBERT, and RoBERTa for anomaly detection using the RT-IoT2022 dataset. Their approach converts structured IoT network traffic into textual representations to enable compatibility with natural language processing models, allowing transformers to leverage contextual relationships within network features. Experimental results demonstrated strong performance, with the BERT-based model achieving low training and validation loss during fine-tuning, indicating effective learning and generalization for attack classification tasks. The authors argue that transformer models provide improved contextual understanding compared to traditional machine learning and conventional deep learning approaches, potentially enabling more intelligent and automated intrusion detection systems for IoT networks. However, **transformer-based approaches typically require substantial computational resources**, which may limit their practicality for real-time deployment in resource-constrained IoT environments.

Collectively, these studies demonstrate the effectiveness of machine learning and deep learning techniques for detecting anomalies in IoT networks. However, they also highlight **persistent challenges, including computational constraints in edge environments, class imbalance in network traffic datasets, and the difficulty of selecting optimal detection thresholds**. These limitations motivate continued research into efficient and interpretable anomaly detection systems for IoT cybersecurity.

2.3 Data Summary

Source Link: https://archive.ics.uci.edu/dataset/942/rt-iot2022

RT-IoT2022 is a network traffic dataset collected from a real-time IoT environment containing both normal device activity and multiple simulated cyberattacks. The dataset includes traffic generated by IoT devices alongside attack scenarios such as brute-force SSH attempts, DDoS attacks (e.g., Hping and Slowloris), and network scanning patterns. Network flows are captured bidirectionally using the Zeek monitoring framework with the Flowmeter plugin, producing structured flow-level features that characterize packet behavior, timing, and protocol dynamics.

In this project, RT-IoT2022 serves as the foundation for developing and evaluating intrusion detection approaches—particularly anomaly detection using autoencoders—aimed at identifying malicious behavior within real-time IoT network traffic.

1) **Predictor (Feature) Variables**: The RT-IoT2022 dataset contains **83 predictor features** describing network traffic flows generated by real-time IoT devices. Rather than representing unrelated variables, these features follow a systematic naming convention built from combinations of directional prefixes and statistical or behavioral suffixes. Each feature captures a measurable property of packet flows—such as volume, timing, payload characteristics, protocol behavior, or connection state—computed either for one traffic direction or for the entire bidirectional flow.

   This structured design allows the dataset to encode network behavior at multiple granularities (packet, flow, and temporal statistics), making it well suited for intrusion detection and anomaly detection tasks. Understanding the prefix–suffix pattern is essential for interpreting the predictors.

   Prefixes (traffic scope / direction):

   - fwd_ (forward): statistics computed for packets traveling from the origin/source to the responder/destination.
   - bwd_ (backward): statistics computed for packets traveling from the responder/destination back to the origin/source.
   - flow_ (flow-level): aggregate statistics computed across the entire bidirectional communication flow.
   - active_ / idle_: metrics describing active transmission periods versus inactive (silent) intervals within a connection.
   - (No prefix): global or contextual attributes describing protocol, ports, or overall flow characteristics.

   Suffixes (measurement type / statistical summary):

   - .min — minimum observed value within the flow.
   - .max — maximum observed value within the flow.
   - .tot — total accumulated value.
   - .avg — arithmetic mean across packets/events.
   - .std — standard deviation measuring variability.
   - _per_sec — rate-based metric normalized by time.
   - _flag_count — count of packets containing specific TCP control flags.
   - _bytes, _pkts, _payload — volume-based measurements describing byte counts, packet counts, or payload sizes.
   - _iat — inter-arrival time statistics between packets.
   - _window_size — TCP window characteristics reflecting transport-layer behavior.

   This prefix–suffix structure enables consistent interpretation across the 83 features while allowing the same behavioral concept (e.g., payload size or timing variability) to be analyzed across traffic directions and statistical summaries.

2) **Outcome (Response) Variable**: Attack_type (categorical)

   - Attack patterns:
     - DOS_SYN_Hping
     - ARP_poisioning
     - NMAP_UDP_SCAN
     - NMAP_XMAS_TREE_SCAN
     - NMAP_OS_DETECTION
     - NMAP_TCP_scan
     - DDOS_Slowloris
     - Metasploit_Brute_Force_SSH
     - NMAP_FIN_SCAN
   - Normal Patterns:
     - MQTT
     - Thing_speak
     - Wipro_bulb_Dataset
     - Amazon-Alexa (**MISSING FROM DATASET**)

3 Exploratory Data Analysis (EDA)

3.1 Initial Data Validation & Skewness Check

1. **Absence of Amazon_Alexa normal traffic**: During dataset initialization and validation as shown in Table 3.1, Table 3.2, and Table 3.3, we observed that the **Amazon_Alexa traffic category was entirely absent** from the available RT-IoT data. According to the original dataset documentation, Amazon_Alexa represents a normal traffic class containing approximately 86,842 observations and plays an important role in balancing the distribution between normal and attack traffic as shown in Table 3.4. Its absence substantially **reduces the volume of normal samples**, creating challenges for the intended anomaly detection framework, particularly for autoencoder training, which relies exclusively on learning normal traffic behavior.

   Because our intrusion detection dataset capture complex relationships across 83 network-flow features, **generating synthetic traffic (data augmentation) that preserves realistic temporal and statistical structure would be difficult** and could introduce artificial patterns. After unsuccessful attempts to obtain the complete dataset, we therefore refine the project scope to focus on IoT device traffic excluding the original voice-assistant (Amazon_Alexa) normal class. We will address the class imbalance for each modeling technique in their respective section.

2. **Categorical Feature Inspection:** From a data type audit in Table 3.5 and Table 3.6, we identified two string-based features within the dataset: `proto` and `service`. Both variables represent low-cardinality categorical attributes rather than identifiers. This suggests that these features encode meaningful protocol and network service information rather than sample-specific identifiers that could introduce data leakage.

3. **Analysis of Feature Skewness:** Feature skewness was evaluated across all numeric variables to identify the need for log transformation. Table 3.7 and Figure 3.1 identified extreme skewness in the dataset with **66 numeric features with skewness over 2.0** indicating that **log transformation is necessary** before modeling. Such skewed distributions are expected in real-world network telemetry, where most flows are small while a minority of flows generate disproportionately large traffic volumes. After log transformation, we can observe in Table 3.8 that **skewness levels drop significantly** and it's safe to assume that long tails are compressed and variances stabilized.

Table 3.1: Summary of feature matrix (X) and target vector (y) dimensions for the dataset, including a sample of feature column names.

Table 3.1

Property	Value
X shape	(123117, 83)
y shape	(123117,)
First 5 X columns	['id.orig_p', 'id.resp_p', 'proto', 'service', 'flow_duration']
y column name	target

Table 3.2: Summary of missing values in the feature matrix (X) and target vector (y), including total counts and missing-value rates.

Table 3.2

dataset	missing_values	missing_rate
X (features)	0	0.000000
y (target)	0	0.000000

Table 3.3: Distribution of target classes in the dataset, including class counts and relative frequencies.

target	count	rate
DOS_SYN_Hping	94659	0.768854
Thing_Speak	8108	0.065856
ARP_poisioning	7750	0.062948
MQTT_Publish	4146	0.033675
NMAP_UDP_SCAN	2590	0.021037
NMAP_XMAS_TREE_SCAN	2010	0.016326
NMAP_OS_DETECTION	2000	0.016245
NMAP_TCP_scan	1002	0.008139
DDOS_Slowloris	534	0.004337
Wipro_bulb	253	0.002055

Table 3.3: Distribution of target classes in the dataset, including class counts and relative frequencies.

target	count	rate
Metasploit_Brute_Force_SSH	37	0.000301
NMAP_FIN_SCAN	28	0.000227

Table 3.4: Binary target label distribution (0=normal, 1=attack).

class	count	rate
0	12507	0.101586
1	110610	0.898414

Table 3.5: Data type summary of feature matrix (X) (excluding target column).

Table 3.5

dtype	count
float64	47
int64	34
str	2

Table 3.6: String features and their unique values.

Table 3.6

string_feature	unique_values
service	10
proto	3

Table 3.7: Summary of skewness statistics across numeric features before log transformation

Table 3.7

total_numerical_features	skewness > 2	skewness > 5	max_skewness	median_skewness
80	65	61	340.497528	29.228034

Table 3.8: Summary of skewness statistics across numeric features after log transformation

Table 3.8

total_numerical_features	skewness > 2	skewness > 5	max_skewness	median_skewness
80	61	11	45.880008	2.639468

Figure 3.1: Distribution of feature skewness before and after log transformation.

3.2 Identifying Class-Separating Features

To better understand which features better distinguish normal and attack traffic, we rank original-scale features using a **Standardized Mean Difference (SMD)** measure aka Cohen's d, a filter-based feature selection approach that quantifies class separability independent of model assumptions (Cohen, 1988).

For each **numeric feature**, we compute a standardized difference between the normal and attack distributions:

$$d = \frac{|\mu_{\text{normal}} - \mu_{\text{attack}}|}{\sigma_{\text{pooled}}}$$

where the pooled standard deviation summarizes variability across both classes:

$$\sigma_{\text{pooled}} = \sqrt{\frac{\sigma^2_{\text{normal}} + \sigma^2_{\text{attack}}}{2}}$$

And:

- μ_{normal} — mean feature value for normal traffic
- μ_{attack} — mean feature value for attack traffic
- σ_{pooled} — pooled standard deviation across both classes
- d — standardized separation (effect size)

This metric quantifies how strongly a feature separates the two binary classes independent of scale. Features with larger effect sizes indicate **greater distributional separation between normal and attack traffic** and are therefore more informative for visualization and downstream modeling analysis as shown in Table 3.9 (Cohen, 1988).

The top-ranked features are selected as **class-separating features** and used in the following exploratory visualizations and analysis.

Table 3.9: Top 10 features by effect size (numeric only)

	Features	Standardized Mean Difference
0	fwd_pkts_payload.min	2.738131
1	fwd_pkts_payload.avg	2.140473
2	fwd_init_window_size	1.666699

Table 3.9: Top 10 features by effect size (numeric only)

	Features	Standardized Mean Difference
3	fwd_pkts_per_sec	1.493684
4	flow_pkts_per_sec	1.493501
5	bwd_pkts_per_sec	1.493306
6	bwd_init_window_size	1.471090
7	payload_bytes_per_second	1.435030
8	active.min	1.054205
9	fwd_subflow_pkts	0.965146

3.3 Analysis of Class-Separating Features

To better understand how normal and attack traffic differ, kernel density estimates (KDEs) were plotted in Figure 3.2 for the top 10 class-separating features identified using standardized mean difference (SMD) above. These visualizations compare the empirical distributions of normal and attack samples for each feature.

1. **Clear Distributional Separation**:

   Across multiple features, attack traffic exhibits distinct statistical behavior compared to normal traffic. In particular:

   - Packet-rate features (`*_pkts_per_sec`, `flow_pkts_per_sec`, `bwd_pkts_per_sec`) show strong magnitude shifts between classes.
   - Payload statistics (`fwd_pkts_payload.min`, `fwd_pkts_payload.avg`, `payload_bytes_per_second`) demonstrate highly concentrated attack distributions with limited overlap.
   - TCP window size features (`*_init_window_size`) reveal substantially different operating ranges between normal and attack traffic.
   - Flow activity and subflow metrics (`active.min`, `fwd_subflow_pkts`) further distinguish attack behavior through tightly clustered value regions.

   In many cases, the **attack distribution forms an extremely narrow peak while normal traffic remains broader and multimodal**, indicating reduced variability within attack-generated flows.

2. **Behavioral Differences in Network Traffic**:

   The observed separation suggests that attacks modify fundamental traffic dynamics rather than subtle statistical properties. Specifically, attack flows tend to exhibit:

   - highly regular packet transmission rates,
   - consistent payload characteristics,
   - constrained TCP window configurations, and
   - structured subflow behavior.

   These patterns are consistent with **automated or scripted attack generation**, which produces repeatable traffic signatures compared to naturally varying user-driven network activity.

3. **Low Overlap Between Classes**:

   Several features display near-disjoint density regions, with attack samples concentrated within narrow intervals while normal traffic spans a wider range of values. This indicates that **individual features already encode strong discriminatory signal**, reducing reliance on complex feature interactions or nonlinear transformations for class separation.

4. **Implications for Downstream Modeling**:

   The presence of strong class separation at the feature level suggests that **linear decision boundaries may capture much of the discriminative structure** present in the dataset. Consequently, a simple supervised baseline model is well justified before introducing more complex representation-learning approaches such as autoencoders.

   Importantly, these findings reflect meaningful behavioral differences between normal and attack traffic patterns captured by the RT-IoT 2022 dataset rather than evidence of data leakage.

   Overall, the exploratory analysis demonstrates that a subset of packet-rate, payload, window-size, and flow-activity features provides substantial statistical separation between classes. These results establish an empirical foundation for subsequent modeling experiments and help explain why relatively simple classifiers may achieve strong performance on this dataset.

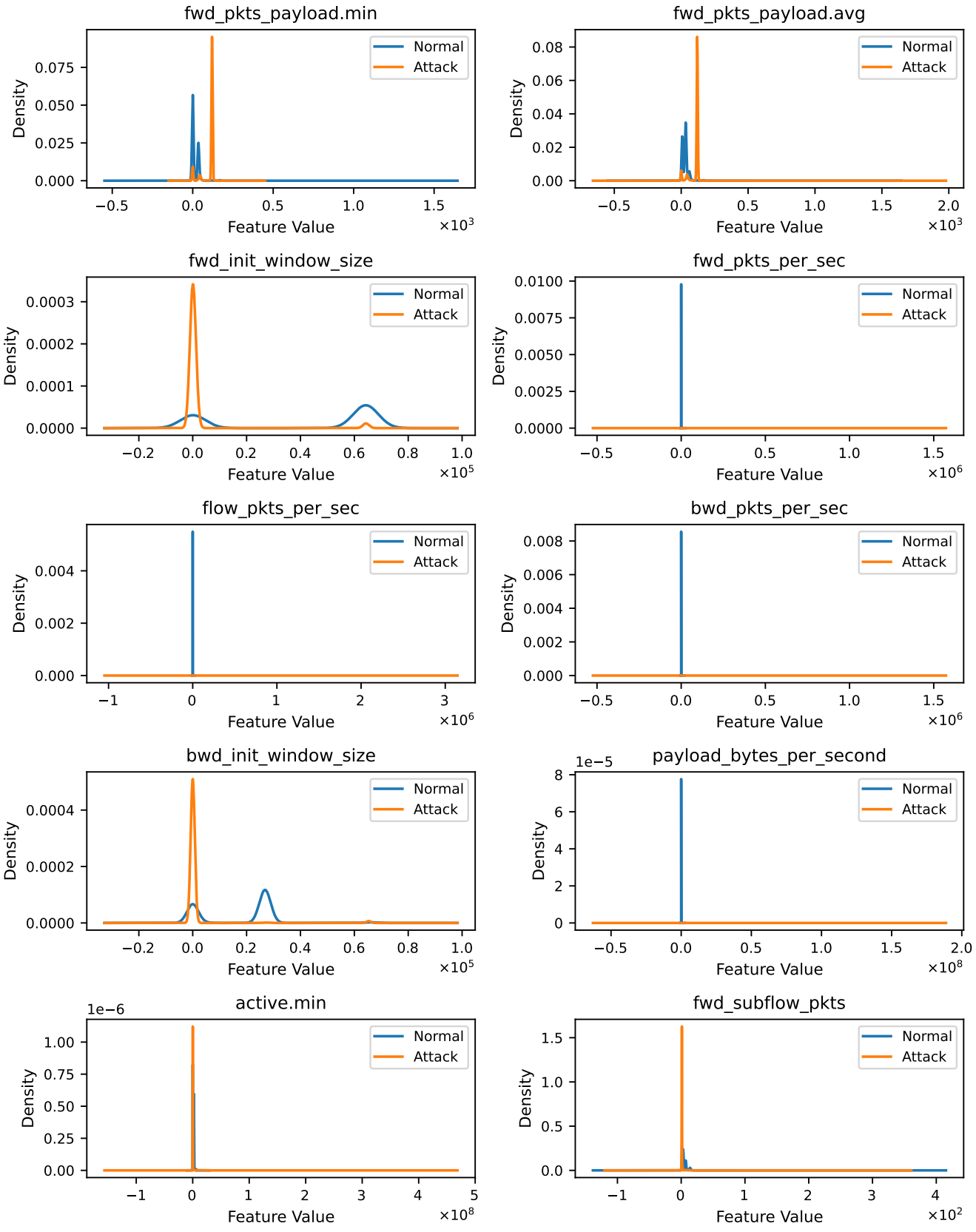

Figure 3.2: KDE plots of top features by effect size

3.4 Correlation & Multicollinearity Analysis

Figure 3.3 presents the Pearson correlation matrix for the top 10 class-separating original-scale numeric features identified above. The heatmap reveals several clear structural patterns that provide insight into multicollinearity and behavioral groupings within the RT-IoT 2022 dataset. Table 3.10 also show that many of their VIF scores are extreme. **68 numeric features carry extreme multicollinearity** (VIF > 10).

These observations enable the following interpretations:

1. **Strong Feature Clustering and Redundancy:**

   The most prominent observation in the heatmap is the presence of distinct blocks of highly correlated features (correlation coefficients approaching ± 1). These clusters indicate that multiple variables capture closely related aspects of the same underlying network behavior rather than independent signals.

   In particular:

   - Packet-rate features (`*_pkts_per_sec`) exhibit near-perfect positive correlation with one another, suggesting they are derived from common traffic counters.
   - Inter-arrival time statistics (`flow_iat.*`, `fwd_iat.*`, `idle.*`) form a dense correlation block, reflecting shared temporal characteristics of network flows.
   - Payload and header-size measurements also display moderate-to-strong correlations, indicating overlapping representations of packet structure.

   This redundancy implies that many high-ranking features encode similar information about traffic intensity and timing dynamics.

2. **Behavioral Grouping of Network Characteristics**:

   The correlation structure naturally separates features into interpretable behavioral categories:

   - **Transmission intensity:** packet rates and payload throughput metrics.
   - **Temporal dynamics:** inter-arrival times, idle durations, and activity statistics.
   - **Protocol and structural properties:** window sizes and header-related features.

   These groupings suggest that attack behavior manifests through coordinated changes across multiple related measurements rather than isolated feature shifts.

3. **Limited Independence Among Top Predictors:** Relatively few features (12) appear statistically independent (VIF $ $ 10). High correlations indicate substantial multicollinearity, meaning that including many correlated variables may not significantly increase predictive power but can increase model complexity and computational cost.

4. **Implications for Feature Selection and Modeling:** The observed correlation structure supports the use of feature reduction prior to deployment. Because many variables measure similar traffic properties, a smaller subset of representative features can likely preserve most predictive information while improving efficiency. For linear models such as Logistic Regression, correlated predictors are partially mitigated through L2 regularization; however, reducing redundancy remains **beneficial for interpretability and resource-constrained deployment** scenarios.

5. **Connection to Dataset Characteristics:** The clustered correlations reflect the engineered nature of network-flow datasets, where multiple statistics are computed from the same packet stream. As a result, these **high correlation does not indicate noise but rather repeated measurements** of shared underlying traffic processes. Overall, the correlation analysis demonstrates that the RT-IoT 2022 dataset contains strong but highly redundant signals, reinforcing the motivation for dimensionality reduction and efficient feature selection in subsequent modeling stages.

Table 3.10: Variance Inflation Factor (VIF) summary for numeric features

VIF Category	Number of Features
High multicollinearity (VIF > 10)	68
Acceptable multicollinearity (VIF $<= 10$)	11

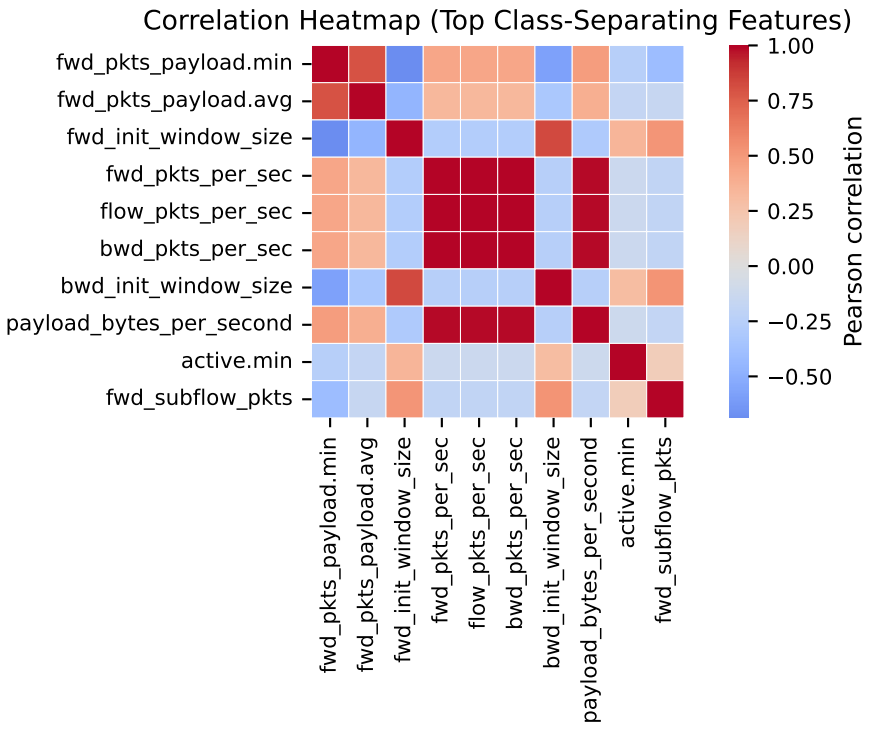

Figure 3.3: Correlation heatmap of top 10 class-separating numeric features (by effect size)

3.5 Mutual Information Analysis

To quantify the predictive relevance of the features, **mutual information (MI)** was computed between each log-scale numeric feature and the binary class labels. MI measures how much knowing a feature reduces uncertainty about class membership, capturing both linear and nonlinear dependencies between network traffic characteristics and attack vs. normal behavior (Peng et al., 2005).

MI quantifies statistical dependence between a feature X and the target class y:

$$I(X;Y) = \sum_{x,y} p(x,y) \log \frac{p(x,y)}{p(x)p(y)}.$$

It can also be interpreted as a reduction in uncertainty:

$$I(X;Y) = H(Y) - H(Y|X),$$

where $H(Y)$ denotes entropy and $H(Y|X)$ represents conditional entropy. A value of $I(X;Y) = 0$ implies statistical independence between X and Y. Because MI captures both linear and nonlinear relationships, it is well suited for network traffic analysis, where attack behavior may alter packet statistics in complex, non-linear ways (Peng et al., 2005).

The top predictive features ranked by mutual information, as shown in Figure 3.4, are:

- fwd_pkts_payload.max
- fwd_subflow_bytes
- fwd_pkts_payload.avg
- flow_pkts_payload.tot
- fwd_pkts_payload.tot
- flow_pkts_payload.max
- flow_pkts_payload.avg
- fwd_last_window_size

From this MI analysis, we can make the following interpretations:

1. **Strong Predictive Signal:**

   All selected features exhibit **high mutual information scores (approximately 0.80–0.88)**, indicating that individual variables contain substantial information about whether a network flow corresponds to normal or attack traffic. The strength of these scores suggests that meaningful class separation exists directly at the feature level, rather than requiring complex nonlinear feature interactions to distinguish traffic behavior. This indicates that the RT-IoT dataset contains strong statistical signatures of attack activity, enabling relatively simple models to exploit clear differences between traffic patterns.

2. **Dominance of Payload-Based Traffic Characteristics:**

   Most of the highest-ranked features are derived from packet payload statistics, including averages, maxima, and total payload sizes. These features capture the volume and structure of transmitted data within a flow, which appears to differ consistently between normal and malicious attack activity. The concentration of payload-related features among the top predictors indicates that attacks modify fundamental transmission behavior. Compared to normal traffic, attack flows tend to exhibit:

   - more uniform/consistent payload sizes,
   - structured packet transmission patterns,
   - and reduced variability across packets within a flow.

   Such regularity is characteristic of **automated attack tools, which generate traffic programmatically rather than through organic user-driven interactions**.

3. **Evidence of Feature Redundancy:**

   Several top features measure closely related quantities (e.g., average, maximum, and total payload statistics). While each individually shows strong predictive power, their similarity suggests substantial redundancy as discussed in our VIF section above. This implies that **increasing the number of included features beyond the highest-ranked subset may provide diminishing returns**, as additional variables often encode overlapping information about the same traffic behavior.

4. **Flow-Level and Transport-Level Behavior Indicators:**

   The feature **fwd__subflow__bytes** captures flow-level transmission behavior, measuring the total number of bytes transmitted within forward-direction subflows. This metric reflects how data is aggregated and transmitted during bursts of communication within a flow. Its high mutual information score suggests that attack traffic often exhibits **distinct burst-level transmission patterns**, potentially due to automated tools generating packets in structured or repetitive sequences.

   The presence of **fwd__last__window__size** among the top predictive features further highlights the role of transport-layer characteristics in distinguishing traffic classes. TCP window size reflects how endpoints regulate data transmission and deviations in this parameter may indicate abnormal communication patterns associated with attack activity or automated traffic generation. Although payload statistics dominate the MI rankings, the inclusion of both fwd__last__window__size and fwd__subflow__bytes demonstrates that transport-layer dynamics and flow-level transmission structure also contribute meaningful information for intrusion detection, complementing the packet-level payload features.

5. **Implications for Downstream Model Selection:**

   The presence of highly informative features imply a strong performance with linear classifiers such as Logistic Regression. Because **substantial class separation exists at the feature level**, complex nonlinear models are not strictly required to achieve strong discrimination performance. From a practical standpoint, these findings suggest that a **compact subset of high-MI features may be sufficient for effective intrusion detection while reducing computational cost**. Overall, the mutual information analysis indicates that payload dynamics and flow-level transmission characteristics constitute the primary statistical signatures distinguishing normal and attack traffic in the RT-IoT dataset.

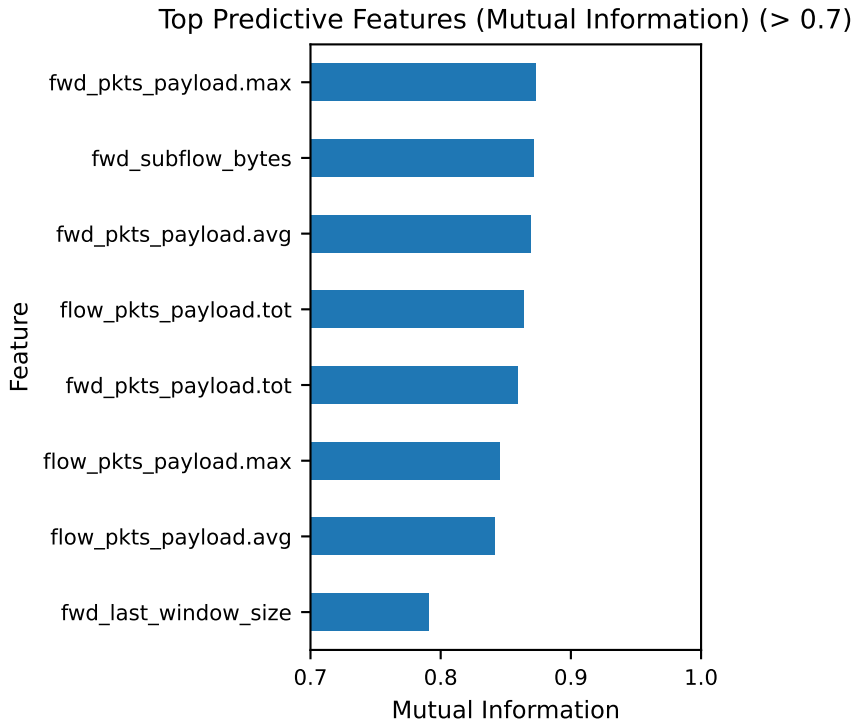

Figure 3.4: Top 8 log-scaled features ranked by mutual information (MI) with the target class label.

3.6 Addressing Categorical Features

The dataset contains several categorical attributes, including service, protocol (proto), and response port identifiers (id.resp_p), which cannot be directly used in distance-based analyses such as PCA or clustering. To incorporate these variables into the modeling pipeline, we performed a structured feature engineering process.

First, the response port feature was **transformed into service-based port groups to reduce its high cardinality** while preserving meaningful network semantics. Ports associated with common application-layer services (e.g., web, DNS, MQTT, SSH) were grouped into dedicated categories, while remaining ports were categorized as either well-known ports or high-numbered ephemeral ports. The resulting distribution shows that FTP-related traffic dominates the dataset (94,672), followed by high ports (8,603), and DNS (7,210) as shown in Table 3.11.

After grouping, the categorical variables (service, proto, and port_group) were **transformed using one-hot encoding**, allowing them to be incorporated into numerical analyses alongside the existing numerical features. The correlation heatmap of the encoded features in Figure 3.5 also reveals several expected relationships.

Strong positive correlations appear between certain service indicators and their corresponding port groups, reflecting consistent mappings between application protocols and typical service ports (e.g., MQTT service with MQTT ports, DNS service with DNS ports). Conversely, negative correlations arise among mutually exclusive categories, particularly between protocol indicators such as TCP and UDP.

Overall, this preprocessing step preserves important categorical information while ensuring compatibility with downstream statistical analysis, dimensionality reduction, and clustering methods in the following section.

Table 3.11: Frequency distribution of grouped response port categories from id.resp_p

	port_group	count
0	ftp	94672
1	high_port	8603
2	dns	7210
3	web	6724
4	mqtt	4142
5	well_known_other	1548
6	ntp	121
7	ssh	62
8	mail	20
9	smtp	15

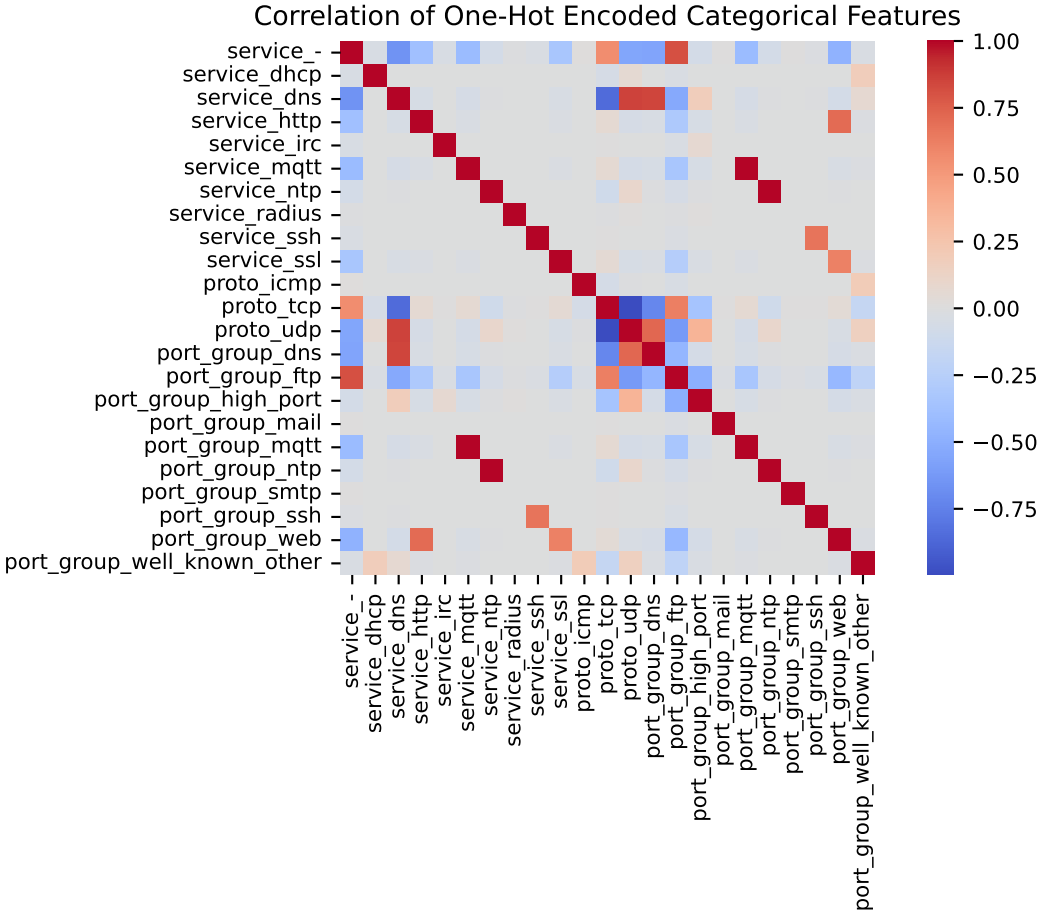

Figure 3.5: Correlation matrix of one-hot encoded categorical features

3.7 Principal Component Analysis (PCA)

Given the 103 features (80 numerical + 23 encoded) after categorical encoding, dimensionality-reduction techniques such as PCA can help analyze the structure of the feature space. Each principal component is a linear combination of the original features and is constructed to capture the maximum possible variance while remaining uncorrelated to other components (Russell & Norvig, 2021).

PCA Mathematical Summary:

We center the Data:

$$\tilde{X}_{ij} = X_{ij} - \mu_j$$

where

$$\mu_j = \frac{1}{n}\sum_{i=1}^{n} X_{ij}$$

Compute the Covariance Matrix:

$$\Sigma = \frac{1}{n-1}\tilde{X}^T\tilde{X}$$

Eigenvalue Decomposition:

$$\Sigma v_k = \lambda_k v_k$$

Project Data onto Principal Components:

$$z_k = \tilde{X}v_k$$

For the first m components:

$$Z = \tilde{X}V_m$$

Derive the Explained Variance Ratio:

$$\text{Explained Variance}_k = \frac{\lambda_k}{\sum_{j=1}^{p}\lambda_j}$$

Calculate the Cumulative Explained Variance:

$$\text{Cumulative Variance}_m = \sum_{k=1}^{m}\frac{\lambda_k}{\sum_{j=1}^{p}\lambda_j}$$

The first principal component (PC1) captures the largest amount of variance in the dataset, the second (PC2) captures the next largest amount while remaining independent of the first, and so on. As a result, PCA can often represent most of the information in a dataset using only a small number of components.

Conceptually, PCA enables **dimensionality reduction** while retaining most of the information, **handles multicollinearity**, can reduce noise when low-variance components are discarded, and allow for visualizations with a small number of principal components to **detect patterns, clusters, and anomalies** (Russell & Norvig, 2021).

After scaling the log-scaled numerical and encoded feature matrix (X), PCA was performed with respect to the binary class labels:

1. The 2-D PC projection in Figure 3.6 explains approximately **52.1% of the total variance** (PC1 $ $ 38.3%, PC2 $ $ 13.7%). While this captures a substantial portion of the variability, the plot shows significant overlap between normal and attack traffic, indicating that the **main variance directions are not strongly aligned with class separation**. This is expected because PCA maximizes variance rather than discriminative power. Several elongated clusters and extreme points are also visible, suggesting **heterogeneous traffic patterns and potential anomalous behaviors** within the dataset.
2. The cumulative explained variance in Figure 3.7 provides stronger insight into the dataset structure. The curve shows that **roughly 20–25 principal components capture about 95% of the variance**, after which additional components contribute minimal new information. This indicates that although the dataset originally contains many features, its effective dimensionality is much lower, reflecting **substantial correlation and shared structure** among network flow features. For modeling, this suggests that moderate dimensionality reduction may retain most of the information while improving model efficiency and reducing multicollinearity.

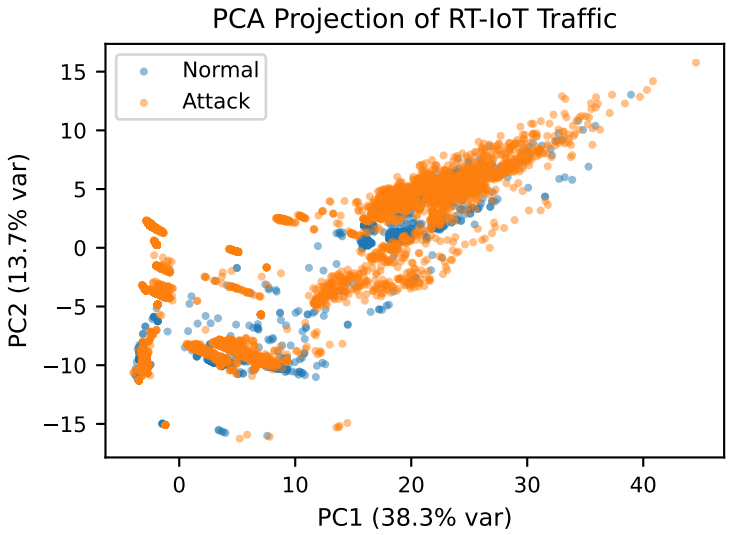

Figure 3.6: PCA projection (2D) on log-scale colored by normal vs attack classes. Explained variance ratios for PC1 and PC2 are indicated in the axis labels.

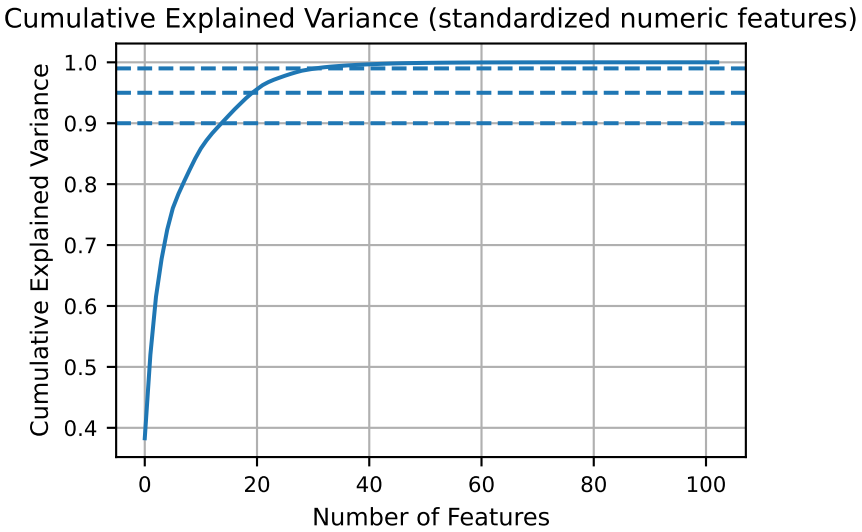

Figure 3.7: Cumulative explained variance of traffic features. Horizontal dashed lines indicate common thresholds (90%, 95%, 99%) for intrinsic dimensionality estimation.

3.8 Clustering Analysis

The last EDA performed is **unsupervised clustering** to explore whether natural groupings exist within the network traffic based on the full set of features. Instead of using normal or attack labels during training, clustering groups observations based solely on similarity in feature space, allowing us to examine the intrinsic structure of the data (Russell & Norvig, 2021).

1. Before clustering, numerical (including encoded) features are **down-sampled with uniform random sampling (n = 25,000) to reduce computational cost of clustering diagnostics** (e.g., Silhouette and Gap statistic) and standardized so that differences in scale do not disproportionately influence distance-based algorithms.
2. We then apply **K-means clustering** to partition the data into groups and evaluate cluster quality using several metrics: **Elbow (inertia), Silhouette score, Calinski–Harabasz (CH), and Gap statistic**.

The Gap statistic compares the within-cluster dispersion to that expected under a reference null distribution (Russell & Norvig, 2021):

$$\text{Gap}(k) = \mathbb{E}_n^*[\log(W_k)] - \log(W_k)$$

where W_k is the within-cluster dispersion for k clusters and $\mathbb{E}_n^*$ denotes expectation under a reference distribution. The standard selection rule is:

$$\text{Choose the smallest } k \text{ such that } \text{Gap}(k) \geq \text{Gap}(k+1) - s_{k+1}$$

In our results from Figure 3.8, the Gap statistic increases monotonically across all tested values of (k) and does not satisfy the stopping criterion. Therefore, the Gap statistic is unable to provide a clear recommendation for the optimal number of clusters.

Instead, the remaining diagnostics collectively suggest **(k = 6)** as a reasonable candidate:

- **Elbow curve:** inertia decreases rapidly until approximately (k=6), after which improvements diminish, indicating reduced marginal benefit from additional clusters.
- **Silhouette score:** a substantial increase occurs at (k=6), followed by only marginal improvements, suggesting improved cluster separation around this value.
- **Calinski–Harabasz score:** while the global maximum occurs at (k=2), a secondary local peak near (k=6) indicates a potentially meaningful cluster structure beyond the trivial two-cluster solution.

Together, these diagnostics suggest that **six clusters capture meaningful structure** in the feature space without over-fragmenting the data. The resulting clusters are visualized in Table 3.12 using the first two principal components similar to the previous section. Although the projection captures only about ~**52% of the total variance (PC1 $ $ 38.2%, PC2 $ $ 13.5%)**, several patterns emerge:

- Several compact clusters near the center of the projection likely correspond to common traffic patterns shared across many observations.
- More isolated clusters and sparse regions suggest rare or specialized traffic behaviors, potentially corresponding to different attack techniques.
- Cluster boundaries do not align clearly with the normal/attack labels, reinforcing the earlier observation that dominant variance patterns primarily reflect **differences among attack behaviors rather than simple benign vs. malicious separation**.

Because the Amazon Alexa normal traffic is absent from the dataset, the class distribution is heavily skewed toward attack observations. As PCA maximizes overall variance and clustering algorithms group observations based on density, both analyses are driven largely by the dominant attack traffic patterns. Consequently, the clusters likely represent **different behavioral modes within attack traffic rather than a clear separation between normal and malicious activity**.

Implications for Modeling:

These exploratory results provide several useful insights for the next modeling phase:

- The dataset exhibits structured but overlapping regions in feature space, suggesting that simple linear separation may be insufficient for reliable classification.
- Clustering reveals multiple behavioral subgroups, indicating that intrusion patterns are heterogeneous i.e., there may not be a consistent pattern in attack traffic.
- Dimensionality-reduction and clustering both confirm that **complex relationships exist across many correlated features**, supporting the use of models capable of capturing nonlinear structure.

Together, these observations motivate the next phase of modeling, where we compare a baseline logistic regression classifier against more expressive models (e.g., autoencoders for anomaly detection) to determine whether reconstruction-based approaches better capture the complex structure of RT-IoT network traffic.

Table 3.12: K-means clustering evaluation metrics across candidate cluster sizes (k = 2–10), including inertia, Silhouette score, Calinski–Harabasz score, and Gap statistic, computed on a standardized random sample (n = 25,000) of log-scale features.

	k	inertia	silhouette	calinski_harabasz	gap	sk
0	2	1.682444e+06	0.708620	12890.283235	4.217992	0.002910
1	3	1.374337e+06	0.667950	10691.725066	4.355002	0.001564
2	4	1.146537e+06	0.669651	10199.105138	4.498236	0.001163
3	5	1.009605e+06	0.682985	9533.971663	4.591595	0.002265

Table 3.12: K-means clustering evaluation metrics across candidate cluster sizes (k = 2–10), including inertia, Silhouette score, Calinski–Harabasz score, and Gap statistic, computed on a standardized random sample (n = 25,000) of log-scale features.

	k	inertia	silhouette	calinski_harabasz	gap	sk
4	6	8.295531e+05	0.739938	10367.232774	4.756633	0.002100
5	7	7.635888e+05	0.745672	9745.161692	4.817793	0.001572
6	8	6.854221e+05	0.748508	9712.373240	4.905605	0.001265
7	9	6.414672e+05	0.762593	9294.344757	4.951872	0.001778
8	10	5.882186e+05	0.765344	9260.525691	5.024099	0.001363

```
Gap-statistic suggested k: None
```

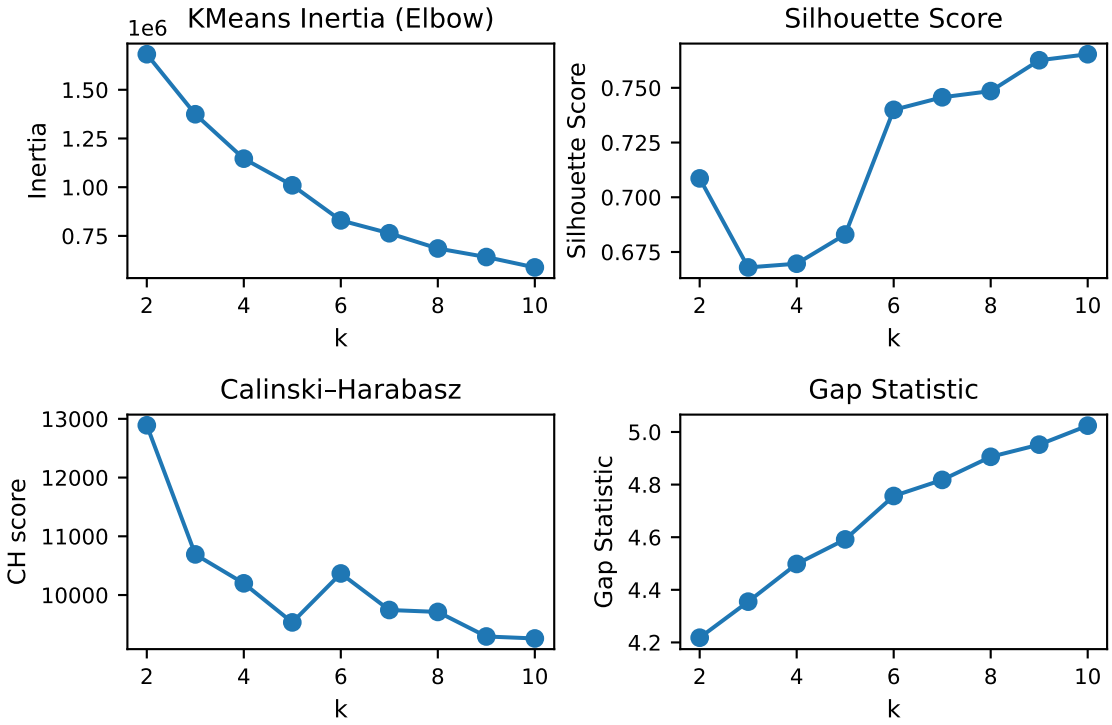

Figure 3.8: K-means clustering diagnostics for selecting the number of clusters (k). The plots show inertia (Elbow method), Silhouette score, Calinski–Harabasz score, and Gap statistic across k = 2–10 using a random sample of 25,000 standardized observations from the log-scale feature matrix.

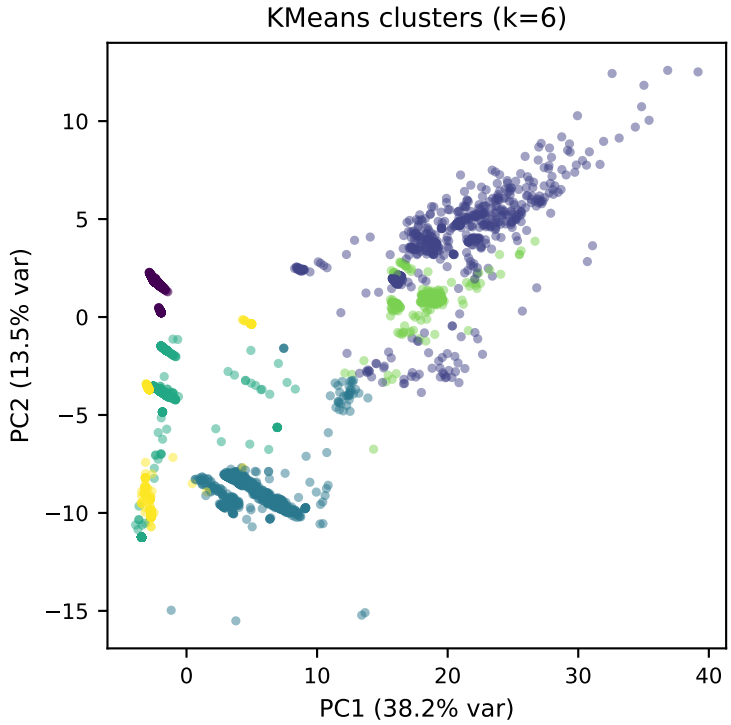

Figure 3.9: K-means clustering results (k = 6) visualized using a 2-D PCA projection of the standardized, log-scale feature matrix. Colors indicate cluster membership, illustrating the structure of traffic behaviors in the first two principal components.

4 Baseline Model: Logistic Regression

4.1 Introduction

4.1.1 Motivation

Before evaluating more complex models such as autoencoders, we establish a **supervised baseline classifier** using Logistic Regression. This model provides a strong statistical reference point because it is:

- Interpretable
- Computationally efficient
- Well-understood theoretically
- A standard benchmark for tabular classification problems

The purpose of this baseline is to measure how well a classical linear model can distinguish between normal and attack network traffic in the RT-IoT 2022 dataset (Hastie et al., 2009).

4.1.2 Model Formulation

Logistic Regression models the probability that an observation belongs to the attack class given a feature vector $x \in \mathbb{R}^d$. The model assumes a linear relationship between features and the log-odds of the outcome:

$$P(y = 1 \mid x) = \sigma(w^\top x + b)$$

where

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

is the sigmoid function, (w) represents model coefficients, and (b) is the intercept.

Equivalently, Logistic Regression models the log-odds (logit) as a linear function:

$$\log \frac{P(y = 1 \mid x)}{P(y = 0 \mid x)} = w^\top x + b$$

4.1.3 Optimization Objective

Model parameters are learned by minimizing the regularized negative log-likelihood (logistic loss):

$$\mathcal{L}(w, b) = -\frac{1}{n} \sum_{i=1}^{n} [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)] + \lambda \|w\|_2^2$$

where

$$p_i = \sigma(w^\top x_i + b)$$

and $\lambda \|w\|_2^2$ represents L2 regularization, which discourages excessively large coefficients and improves generalization.

The optimization is solved numerically using gradient-based methods (LBFGS in scikit-learn) (Hastie et al., 2009).

4.2 Baseline Model & Model Selection

We began with the log-scaled feature matrix from the EDA section and a binary target label indicating normal versus attack traffic. The data were split into training, validation, and test partitions, with the **training set further stratified and downsampled to 25% of its original size** for computational feasibility while preserving the class distribution, as shown in Table 4.1. This downsampling choice reduced the cost of repeated model fitting during feature selection without altering the validation and test sets used for final assessment.

Table 4.1: Stratified train, validation, and test split. The training set is stratified and downsampled to 25% of its original size to reduce training time while preserving the class distribution. Validation and test sets remain unchanged for unbiased model evaluation.

	split	rows	attack_rate
0	train	21545	0.898399
1	val	18468	0.898419
2	test	18468	0.898419

To reduce dimensionality and stabilize the baseline logistic regression workflow, we first fit an **L1-regularized (Lasso)** logistic regression pipeline with scaling, then applied **bidirectional stepwise selection using the BIC criterion** on the reduced feature set from Lasso, again using scaled predictors for numerical stability. The accepted stepwise path is summarized in Table 4.2. Starting from the intercept-only model, BIC dropped sharply from ~14,168.8 to 1,779.2, yielding a final 9-feature subset:

- `fwd_iat.min`
- `flow_iat.min`
- `bwd_iat.avg`
- `active.std`
- `flow_pkts_per_sec`
- `bwd_pkts_payload.min`
- `fwd_header_size_min`
- `id.orig_p`
- `flow_ECE_flag_count`

Practically, this indicates that a compact set of **timing, packet-rate, payload, header, port, and flag-based variables captured most of the predictive signal** needed by this baseline linear classifier (for comparison with autoencoder models).

Table 4.2: Bidirectional stepwise BIC feature-selection path starting from the intercept-only logistic regression model.

Table 4.2

step	action	feature	bic	delta_bic	n_selected
0	start	nan	14168.800776	nan	0
1	add	fwd_iat.min	3776.158982	-10392.641794	1
2	add	flow_iat.min	2898.127040	-878.031942	2
3	add	bwd_iat.avg	2477.022487	-421.104553	3
4	add	active.std	2132.621888	-344.400599	4
5	add	flow_pkts_per_sec	1985.317551	-147.304337	5
6	add	bwd_pkts_payload.min	1917.826027	-67.491524	6
7	add	fwd_header_size_min	1813.276007	-104.550020	7
8	add	id.orig_p	1788.617733	-24.658273	8
9	add	flow_ECE_flag_count	1779.190261	-9.427472	9

To directly compare them, we then fit and tuned three logistic regression variants: the full/saturated baseline model, the LASSO-selected subset, and the stepwise-selected subset. The classification threshold for each model was **tuned on the validation set to maximize F1 for the attack class**, and final performance was compared on the held-out test set using accuracy, precision, recall, ROC-AUC, PR-AUC, and F1, as summarized in Table 4.3. Confusion matrices were also retained in Table 4.4 to make the false-positive versus false-negative tradeoff explicit. This evaluation setup provided a consistent basis for comparing discrimination performance across all three models with **ROC-AUC and PR-AUC being the primary metrics of our evaluation** for imbalanced anomaly detection systems. This again highlights that a compact set of features in the stepwise selected model captured most of the predictive signal.

Table 4.3: Validation-tuned and test-set performance comparison of the baseline, LASSO-reduced, and stepwise-selected logistic regression models.

Table 4.3

model	n_features	threshold	val_f1	test_roc_auc	test_pr_auc	test_accuracy	test_precision	test_recall	test_f1
baseline_lr	103	0.070000	0.996806	0.999055	0.999898	0.994964	0.997527	0.996866	0.997197
lasso_lr	23	0.060000	0.996836	0.996772	0.999587	0.994477	0.997706	0.996143	0.996924
stepwise_lr	9	0.050000	0.996625	0.996560	0.999626	0.994423	0.997466	0.996324	0.996894

Table 4.4: Confusion matrices for the baseline, LASSO-reduced, and stepwise-selected logistic regression models evaluated on the held-out test set using the validation-tuned classification thresholds.

Table 4.4

model	TN	FP	FN	TP
baseline_lr	1835	41	52	16540
lasso_lr	1838	38	64	16528
stepwise_lr	1834	42	61	16531

Because the **stepwise-selected model was the most parsimonious and best-supported specification**, we refit it in statsmodels for diagnostic analysis. The final fit summary in Table 4.5 showed log-likelihood of approximately -839.71, AIC of ~1,699.41, BIC of ~1,779.19, and successful convergence. This is an important practical result, the final selected model converged cleanly and produced finite likelihood-based fit statistics.

The coefficient summary in Table 4.6 showed generally plausible coefficient magnitudes and directions. In particular, several timing-related features such as flow_iat.min and bwd_iat.avg had negative coefficients, while flow_pkts_per_sec and bwd_pkts_payload.min had positive coefficients, indicating that packet timing and traffic intensity were key discriminators between normal and attack traffic flows. Most coefficients were stable and interpretable. The main exception was flow_ECE_flag_count, which exhibited an unusually large standard error despite a finite coefficient estimate. This pattern is consistent with a sparse or low-variance indicator-like feature and **should be interpreted cautiously, though it was still retained by the BIC search as a useful predictor**.

Table 4.5: Final statsmodels logistic regression fit using the stepwise-selected feature subset.

Table 4.5

metric	value
n_train_rows	21545
n_features	9
log_likelihood	-839.705635
aic	1699.411271
bic	1779.190261
converged	True

Table 4.6: Coefficient estimates and standard errors for the final stepwise-selected logistic regression model fitted with statsmodels.

	Coef.	Std.Err.	z	P>|z|	[0.025	0.975]
const	6.052876	2.064253e+06	2.932235e-06	9.999977e-01	-4.045855e+06	4.045867e+06
fwd_iat.min	-0.376742	6.298874e-02	-5.981099e+00	2.216377e-09	-5.001975e-01	-2.532862e-01
flow_iat.min	-1.581502	6.497773e-02	-2.433915e+01	7.553566e-131	-1.708856e+00	-1.454148e+00
bwd_iat.avg	-1.946933	1.073298e-01	-1.813971e+01	1.548334e-73	-2.157295e+00	-1.736570e+00
active.std	0.690525	4.059956e-02	1.700819e+01	7.140937e-65	6.109513e-01	7.700987e-01
flow_pkts_per_sec	1.182500	1.590564e-01	7.434472e+00	1.049867e-13	8.707555e-01	1.494245e+00
bwd_pkts_payload.min	1.123098	9.728699e-02	1.154418e+01	7.899516e-31	9.324191e-01	1.313777e+00
fwd_header_size_min	0.940646	9.801824e-02	9.596640e+00	8.259286e-22	7.485335e-01	1.132758e+00
id.orig_p	0.563363	8.072033e-02	6.979199e+00	2.968678e-12	4.051543e-01	7.215722e-01
flow_ECE_flag_count	-6.466269	8.847643e+07	-7.308465e-08	9.999999e-01	-1.734106e+08	1.734106e+08

4.3 Assumption Checking & Validation

Multicollinearity improved substantially after LASSO reduction and stepwise refinement. The final VIF table in Table 4.7 showed that most predictors had low VIF values, with only fwd_iat.min and bwd_iat.avg slightly above 5. This is acceptable in context because both variables represent related inter-arrival time behavior and some correlation is expected in flow-based network telemetry. More importantly, no predictor approached the level typically associated with severe multicollinearity as we observed in our previous EDA, so the final model appears structurally stable.

Table 4.7: Variance inflation factors (VIF) for the final stepwise-selected logistic regression predictors.

Table 4.7

feature	VIF
bwd_iat.avg	5.775211
fwd_iat.min	5.499405
bwd_pkts_payload.min	3.435348

feature	VIF
fwd_header_size_min	3.012110
flow_iat.min	2.857306
flow_pkts_per_sec	1.821093
active.std	1.346072
id.orig_p	1.065869
flow_ECE_flag_count	1.025462
const	1.000000

Influence diagnostics also supported the adequacy of the final model. In Figure 4.1, most observations had Cook's distance values near zero, with only a small number of moderate spikes. The summary metrics in Table 4.8 showed a maximum Cook's distance of roughly 0.148 and a near-zero mean, indicating that no individual observation exerted excessive influence on the fitted model. This suggests **the model is not being driven by a handful of anomalous flows**, which is especially important in intrusion detection settings where rare but extreme traffic patterns are common.

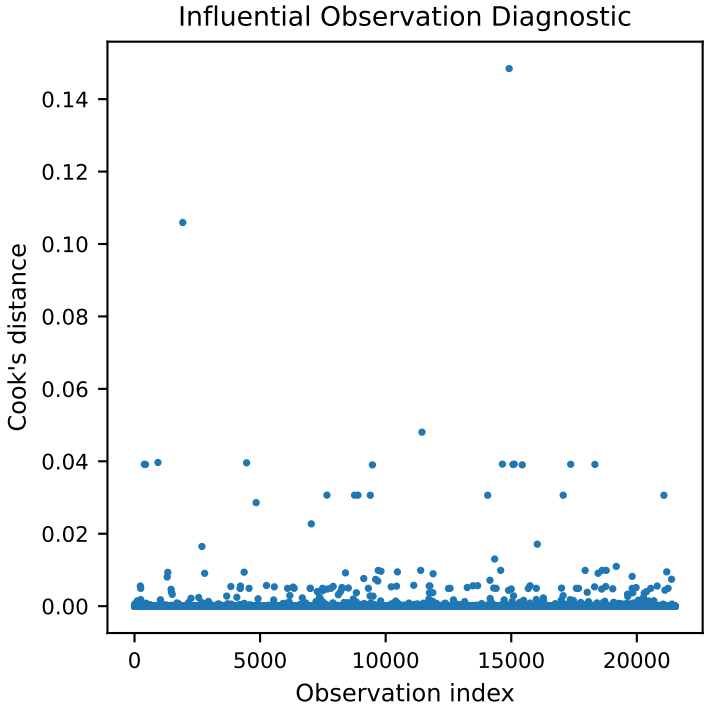

Figure 4.1: Cook's distance for observations in the final logistic regression model.

Table 4.8: Cook's distance summary for the final logistic regression model showing the maximum value, mean Cook's distance, and the conventional 4/n influence threshold.

	max_cooks	mean_cooks	threshold
0	0.148443	0.000083	0.000186

We next assessed the linearity of the logit assumption visually in Figure 4.2. The selected predictors generally exhibited monotonic, approximately linear relationships with the fitted logit, with no strong U-shaped or systematically curved patterns. Some predictors showed discrete clustering, which is expected for network-derived count and header fields, and flow_ECE_flag_count again appeared sparse. Overall, the visual evidence suggests that the **linearity assumption is reasonable** for this specification.

Finally, the residual diagnostics indicated acceptable model fit. The deviance residual histogram in Figure 4.4 was sharply centered near zero with symmetric but somewhat heavy tails, while the deviance residual scatterplot in Figure 4.3 showed a dense band around zero and no obvious structure across observation range. A few larger residuals were present, but there was **no evidence of broad misspecification or systematic underfit**.

Overall, the BIC stepwise-selected baseline logistic regression model is statistically defensible and practically informative. The final stepwise model converged successfully, reduced the feature space to nine interpretable predictors, improved multicollinearity relative to the larger candidate set, showed no problematic influence behavior, satisfied the linearity-of-the-logit assumption reasonably well, and produced acceptable residual diagnostics. This gives us **a strong classical baseline against which the upcoming autoencoder-based anomaly detection models can be compared**.

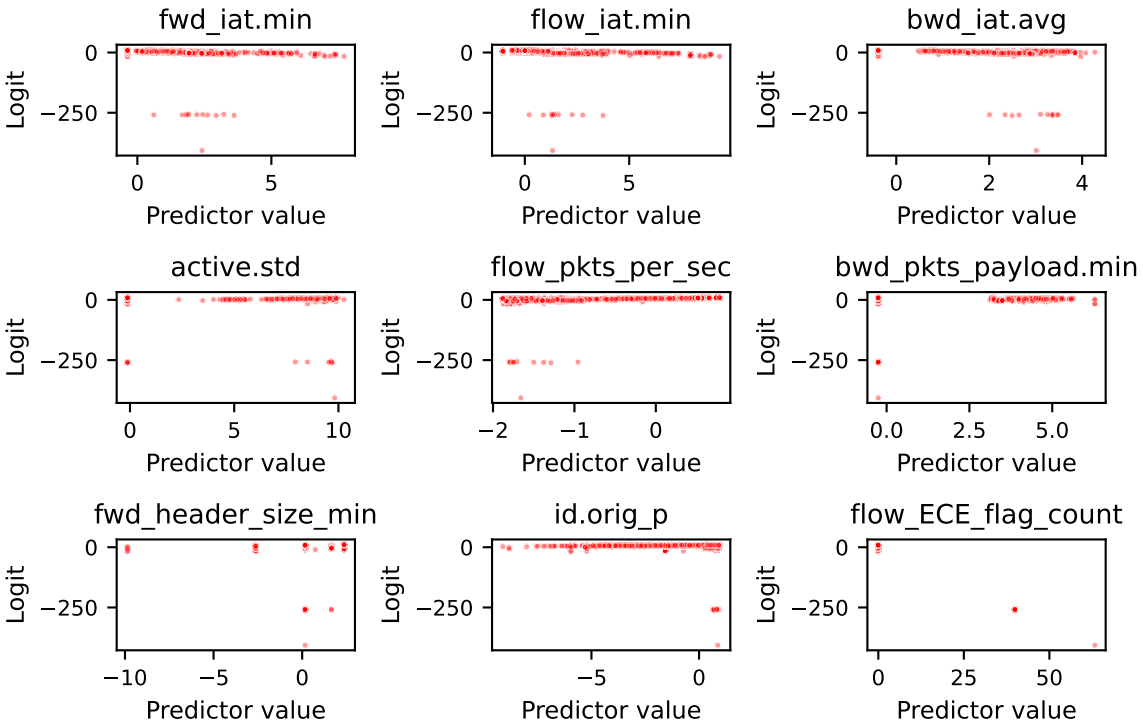

Figure 4.2: Scatterplots of predictor values versus the logit of predicted probabilities used to visually assess the linearity-of-the-logit assumption for the final logistic regression model.

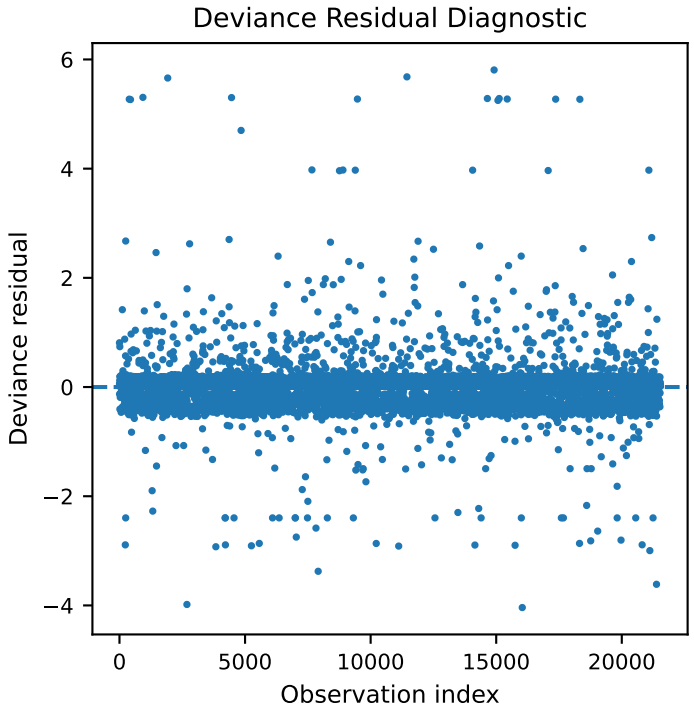

Figure 4.3: Deviance residuals for the final logistic regression model.

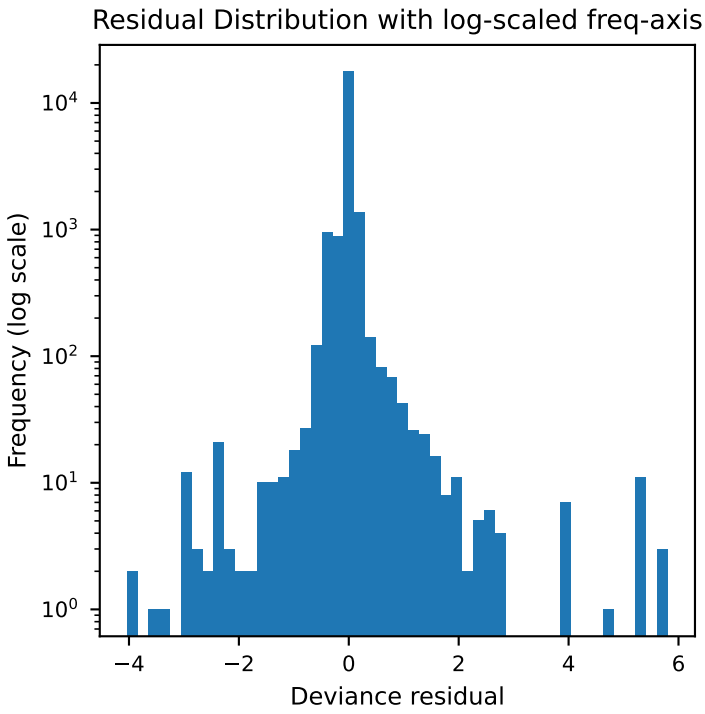

Figure 4.4: Distribution of deviance residuals for the final logistic regression model (log-scaled frequency axis).

5 Autoencoder (AE) Models

5.1 Introduction

Intrusion detection in IoT networks presents unique challenges due to the heterogeneity of devices, communication protocols, and traffic patterns. Traditional supervised classifiers rely on labeled examples of both normal and attack traffic activity. However, in many real-world cybersecurity settings, obtaining representative attack samples is difficult because new attack patterns continually emerge.

Autoencoders provide a useful alternative by learning the structure of normal network behavior without requiring labeled attack traffic data. Instead of directly predicting class labels, an autoencoder learns to reconstruct input observations that follow the normal traffic distribution. When the model encounters anomalous or malicious traffic patterns that deviate from this learned structure, the reconstruction error tends to increase. This property allows **reconstruction error to serve as an anomaly score for intrusion detection**.

In this section, autoencoders are trained exclusively on normal network traffic from the RT-IoT 2022 dataset. **Observations that produce large reconstruction errors during inference are classified as anomalous** and interpreted as potential attacks.

5.1.1 Model Formulation

An autoencoder is a neural network designed to learn a compressed representation of input data and reconstruct the original input from this representation. It consists of two components:

- Encoder: maps the input feature vector into a lower-dimensional latent representation.
- Decoder: reconstructs the original input from the latent representation.

$$x \in \mathbb{R}^d$$

where d represents the number of features in the dataset.

The encoder transforms the input into a latent representation z:

$$z = f_\theta(x)$$

where $f_\theta(\cdot)$ denotes the encoder network parameterized by weights θ.

The decoder reconstructs the input:

$$\hat{x} = g_\phi(z)$$

where $g_\phi(\cdot)$ represents the decoder network parameterized by weights ϕ.

Combining both operations gives the full autoencoder mapping:

$$\hat{x} = g_\phi(f_\theta(x))$$

The encoder compresses the information in the input into a lower-dimensional representation, while the decoder attempts to reconstruct the original feature vector from that representation.

5.1.2 Training Objective

The autoencoder is trained using the **mean squared reconstruction error** over the training dataset consisting only of normal traffic observations.

For a dataset with N samples, the optimization objective is

$$\min_{\theta,\phi} \frac{1}{N} \sum_{i=1}^{N} \|x_i - \hat{x}_i\|^2$$

where

$$\hat{x}_i = g_\phi(f_\theta(x_i))$$

This objective encourages the model to learn latent representations that accurately capture the structure of normal traffic. Because the model is only exposed to benign samples during training, it becomes specialized at reconstructing patterns consistent with normal network behavior.

5.1.3 Reconstruction Error as an Anomaly Score

After training, the reconstruction error for a new observation is computed as

$$s(x) = \|x - \hat{x}\|^2$$

where $s(x)$ represents the anomaly score for observation x.

Intuitively:

- **Normal traffic** should produce small reconstruction errors because the model has learned similar patterns during training.
- **Attack traffic** often produces larger reconstruction errors because the feature patterns differ from those observed during training.

Therefore, reconstruction error serves as a continuous anomaly score used to detect suspicious traffic.

5.1.4 Threshold Selection via Validation F1 Score:

To convert anomaly scores into binary predictions, a threshold τ must be selected such that

$$\hat{y} = \begin{cases} 1 & \text{if } s(x) > \tau \\ 0 & \text{otherwise} \end{cases}$$

where

- $\hat{y} = 1$ indicates an attack
- $\hat{y} = 0$ indicates normal traffic.

The threshold τ is selected using the validation dataset by maximizing the **F1 score**, defined as

$$F1 = \frac{2(\text{Precision} \cdot \text{Recall})}{\text{Precision} + \text{Recall}}$$

where

$$\text{Precision} = \frac{TP}{TP + FP}$$

and

$$\text{Recall} = \frac{TP}{TP + FN}$$

with TP, FP, and FN denoting true positives, false positives, and false negatives respectively.

The optimal threshold is chosen as

$$\tau^* = \arg\max_{\tau} F1(\tau)$$

5.1.5 Why F1 Score is Used

The RT-IoT dataset exhibits a strong imbalance between attack and normal traffic, with attack observations comprising a large portion of the dataset. In such settings, metrics such as **accuracy can be misleading** because a model may achieve high accuracy while failing to detect rare or minority classes effectively.

The F1 score balances precision and recall, making it particularly suitable for intrusion detection tasks:

- **Precision** penalizes false alarms by measuring the proportion of predicted attacks that are actually malicious.
- **Recall** measures the ability of the model to correctly detect attacks.

By maximizing the F1 score during threshold selection, the model achieves a balance between detecting malicious activity and minimizing false positives, which is critical for practical intrusion detection systems.

5.2 Baseline AE Model

5.2.1 Data Preparation

To evaluate the autoencoder models, we first imported the processed but unscaled feature matrix produced during the EDA stage. This matrix consists of both numeric features and one-hot encoded categorical variables, resulting in a fully numeric design matrix suitable for neural network training. The corresponding binary target labels were also imported, where normal traffic is represented by $y = 0$ and attack traffic by $y = 1$.

The dataset was then partitioned using a stratified train, validation, and test split in order to preserve the original class proportions across all subsets. Importantly, **no downsampling was performed** during this step. Maintaining the full dataset ensures that the training procedure has access to the largest possible set of normal/benign observations, which is critical for learning a robust representation of normal network behavior. The resulting split proportions are summarized in Table 5.1.

Because the autoencoder is trained in an unsupervised anomaly detection setting, the model should only learn from benign examples. Therefore, after the stratified split was performed, **all attack observations were removed from the training set**, leaving only normal traffic for model fitting. The validation and test sets retain both normal and attack observations so that model performance can be evaluated in a realistic detection scenario.

Table 5.1: Stratified train, validation, and test split preserving the original class distribution. Autoencoder models are trained using only normal traffic (y=0) from the training set, while validation and test sets retain both normal and attack observations for anomaly detection evaluation.

	split	rows	attack_rate
0	train_all	86181	0.898411
1	train_normal	8755	0.000000
2	val	18468	0.898419
3	test	18468	0.898419

5.2.2 Feature Scaling and Tensor Conversion

Neural networks are sensitive to the scale of input features. To ensure stable training, we applied feature standardization using the training data. Specifically, a scaler was fitted on the normal training observations and then applied to the validation and test features using the same parameters. This prevents information leakage from the evaluation sets into the training process. After scaling, the feature matrices were converted into **PyTorch tensors with 32-bit floating point precision (float32)**, which is the standard datatype used for neural network training in PyTorch.

5.2.3 Baseline Autoencoder Architecture

The baseline autoencoder architecture was designed as a **fully connected symmetric encoder–decoder network**, with the configuration summarized in Table 5.2. The encoder compresses the input feature vector into a lower-dimensional latent representation, while the decoder attempts to reconstruct the original input from this compressed representation. This architecture introduces a bottleneck latent space, forcing the model to learn compact representations of normal traffic patterns. If an observation differs substantially from the learned distribution, the decoder will struggle to reconstruct it accurately, resulting in a larger reconstruction error that can signal anomalous behavior.

Table 5.2: Baseline autoencoder architecture configuration. The encoder compresses the 103-dimensional input feature space through hidden layers of 64 and 32 units into a 16-dimensional latent representation, while the decoder mirrors this structure to reconstruct the original input. This gradual compression introduces a bottleneck that encourages the model to learn compact representations of normal network traffic patterns.

	input_dim	hidden_dim_1	hidden_dim_2	latent_dim	dropout
0	103	64	32	16	0.0

5.2.4 Training Procedure

The normal training observations were wrapped into a **mini-batch data loader**, enabling the model to process subsets of data during each optimization step. Mini-batch training improves computational efficiency and stabilizes gradient updates compared to full-batch optimization. The autoencoder was trained using the mean squared error (MSE) reconstruction loss as explained in the introduction. This loss function directly aligns with the goal of autoencoder training: minimizing the reconstruction error of normal traffic patterns.

Model parameters were optimized using the **Adam optimizer**, which is a widely used adaptive gradient method that combines the advantages of momentum-based optimization and adaptive learning rates. Adam is particularly well suited for neural networks because it can efficiently handle noisy gradient estimates and typically converges faster than standard stochastic gradient descent. The model was trained for **30 epochs**, during which the average training reconstruction error steadily decreased, indicating that the network successfully learned to reproduce normal traffic behavior. The evolution of the training loss over epochs is shown in Figure 5.1.

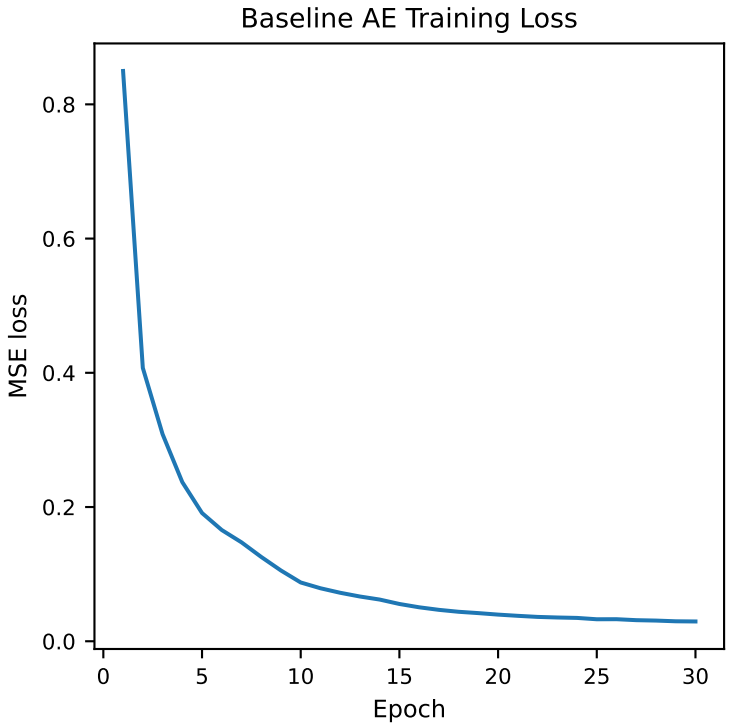

Figure 5.1: Training mean squared reconstruction loss for the baseline autoencoder across 30 epochs using normal traffic only. The decreasing loss indicates that the model progressively improves its ability to reconstruct normal network behavior, which forms the basis for anomaly detection via reconstruction error.

5.2.5 Reconstruction Error and Threshold Selection

After training, the autoencoder was applied to the validation and test sets to compute the **reconstruction error** for each observation. Because the model has only seen normal traffic during training, benign observations are expected to produce small reconstruction errors, while malicious traffic typically produces larger errors.

To convert reconstruction errors into binary predictions, a decision threshold was selected using the validation set. Specifically, a range of candidate thresholds was evaluated and the value that **maximized the validation F1 score** was chosen. The F1 score balances precision and recall, making it particularly suitable for imbalanced intrusion detection datatsets where both missed attacks and excessive false alarms are undesirable.

5.2.6 Baseline Model Performance

Using the optimal threshold derived from the validation set, the baseline autoencoder was evaluated on the held-out test set. Performance metrics including accuracy, precision, recall, F1 score, ROC-AUC, and PR-AUC are reported in Table 5.3. These metrics collectively assess the model's ability to correctly identify attack traffic while minimizing false positives.

The resulting confusion matrix, shown in Table 5.4, provides a detailed breakdown of classification outcomes. The matrix illustrates how many attack samples were correctly detected, how many benign samples were mistakenly flagged as anomalies, and how often the model produced correct normal predictions.

Overall, the baseline autoencoder demonstrates **strong anomaly detection capability**, indicating that reconstruction error effectively separates normal and malicious traffic patterns in the dataset.

Table 5.3: Baseline autoencoder anomaly detection performance. The anomaly threshold was selected on the validation set by maximizing the F1 score, and final evaluation metrics (accuracy, precision, recall, F1, ROC-AUC, and PR-AUC) are reported on the held-out test set.

Table 5.3

threshold	n_features	val_f1	test_accuracy	test_precision	test_recall	test_f1	test_roc_auc	test_pr_auc
0.050757	103	0.995129	0.991986	0.993636	0.997469	0.995549	0.996057	0.999491

Table 5.4: Confusion matrix for the baseline autoencoder evaluated on the test set using the threshold selected from validation F1 maximization. The matrix summarizes correct and incorrect classifications of normal and attack traffic, illustrating the model's ability to detect malicious flows while maintaining a relatively low false-positive rate for benign traffic.

	Pred Normal	Pred Attack
Actual Normal	1770	106
Actual Attack	42	16550

5.3 Quantized Autoencoder Variants (QAE-f16 and QAE-u8)

After establishing the baseline autoencoder, we investigated whether similar anomaly detection performance could be maintained under reduced numerical precision, which is particularly relevant for deployment on resource-constrained IoT and edge devices. Two quantized variants were evaluated: a **half-precision autoencoder (QAE-f16)** and an **8-bit dynamically quantized autoencoder (QAE-u8)**. Rather than retraining new models, both variants were obtained by converting the trained baseline autoencoder to lower-precision representations and repeating the inference and evaluation pipeline.

The first variant, QAE-f16, converts the model parameters from standard 32-bit floating point (FP32) precision to 16-bit floating point (FP16). **Half precision reduces the memory footprint of the model by approximately 50%**, while typically preserving most of the numerical fidelity required for neural network inference. Because the model architecture and learned weights remain unchanged aside from precision, the FP16 model can be evaluated using the same inference pipeline as the baseline autoencoder.

The second variant, QAE-u8, applies dynamic 8-bit quantization to the linear layers of the trained autoencoder. In this setting, model weights are stored internally as 8-bit integers (INT8), and activations are dynamically quantized during inference. Dynamic quantization significantly reduces model memory requirements and can improve CPU inference efficiency, particularly on edge hardware devices. Compared with FP32 weights, INT8 quantization **reduces the storage required per parameter by approximately 75%**, making it an attractive option for lightweight deployment.

Once the baseline model was converted into the QAE-f16 and QAE-u8 variants, we repeated the same inference procedure used for the baseline autoencoder. Reconstruction errors were computed for the validation and test sets, and a decision threshold was selected by maximizing the validation **F1 score** using the same threshold search procedure from the previous section. The resulting thresholds, along with the final evaluation metrics, are summarized in Table 5.5.

One notable observation is that the QAE-u8 model produced a slightly shifted reconstruction error threshold compared with the baseline and QAE-f16 models. However, despite this shift in threshold magnitude, the overall detection performance remained remarkably stable. Across all three models, metrics such as *ccuracy, precision, recall, F1 score, ROC-AUC, and PR-AUC exhibit only minimal deviations, indicating that the **quantized representations preserve the essential structure learned by the original autoencoder**.

The comparative performance of the three models is visualized in Figure 5.2, which shows that both quantized variants achieve nearly identical test-set performance to the baseline autoencoder. These results suggest that reduced-precision autoencoders can maintain strong anomaly detection capability even when model weights are stored with significantly lower numerical precision.

Table 5.5: Comparison of baseline and quantized autoencoder models. Metrics include validation F1 used for threshold selection and test-set performance metrics (accuracy, precision, recall, F1, ROC-AUC, and PR-AUC). Despite reduced numerical precision, both QAE-f16 and QAE-u8 achieve performance nearly identical to the baseline autoencoder, with only minor deviations in reconstruction threshold.

Table 5.5

model	threshold	n_features	val_f1	test_accuracy	test_precision	test_recall	test_f1	test_roc_auc	test_pr_auc
Baseline AE	5.076e-02	1.030e+02	9.951e-01	9.920e-01	9.936e-01	9.975e-01	9.955e-01	9.961e-01	9.995e-01
QAE-f16	5.076e-02	1.030e+02	9.951e-01	9.920e-01	9.936e-01	9.975e-01	9.955e-01	9.961e-01	9.995e-01
QAE-u8	9.134e-02	1.030e+02	9.948e-01	9.922e-01	9.935e-01	9.978e-01	9.957e-01	9.960e-01	9.995e-01

From a deployment perspective, the most compelling advantage of quantization is the reduction in **model storage requirements**. As illustrated in Figure 5.4, decreasing the parameter precision from 32-bit (baseline) to 16-bit (QAE-f16) and 8-bit (QAE-u8) progressively lowers the number of bits required to store each weight. This reduction directly translates to smaller model artifacts and potentially faster inference on CPU-bound edge devices. The ability of **QAE-u8 to retain near-baseline detection performance while dramatically reducing precision requirements highlights its practical value for IoT environments**, where computational resources and memory are often limited.

Finally, the reconstruction error distributions for normal and attack traffic across all three models are shown in Figure 5.3. These distributions demonstrate that the separation between benign and malicious traffic remains clearly visible even after quantization. This stability explains why the detection metrics remain largely unchanged despite the reduction in numerical precision.

The results in Table 5.6 indicate that all evaluated models achieve very strong classification performance on the RT-IoT 2022 dataset. Given the **class imbalance between normal and attack traffic**, greater emphasis should be placed on **ROC-AUC and PR-AUC**, which provide a more reliable assessment of detection performance across decision thresholds. All models achieve extremely high ROC-AUC values (0.996), indicating excellent separability between normal and malicious traffic. The logistic regression models also achieve **very high PR-AUC values (0.9996)**, while the autoencoder models achieve **PR-AUC values near 0.9995**, reflecting strong precision–recall tradeoffs for the attack class. These results suggest that **the dataset contains highly separable patterns between normal and attack traffic**. The baseline logistic regression model already performs exceptionally well, achieving a test ROC-AUC of **0.996055**, a

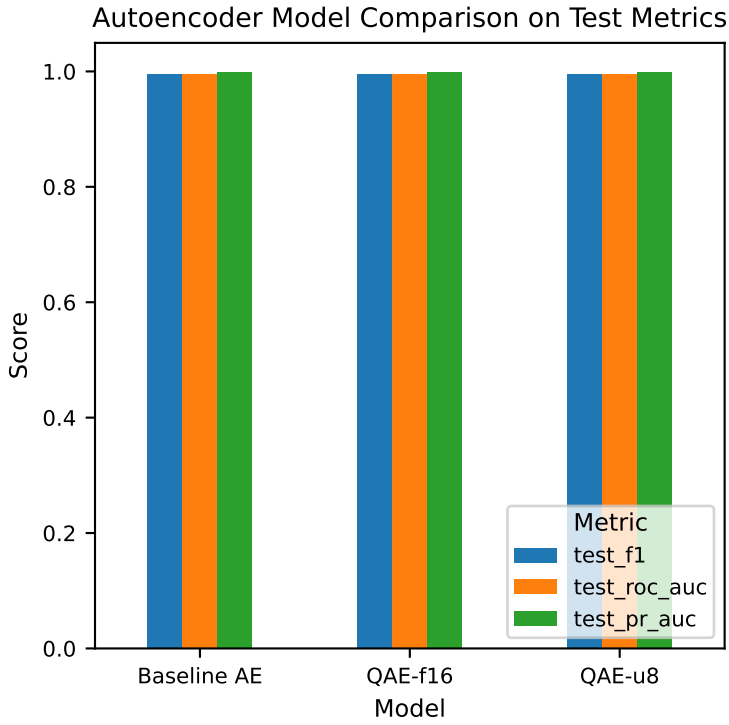

Figure 5.2: Comparison of baseline and quantized autoencoder models on key test-set anomaly detection metrics. All three models achieved strong performance.

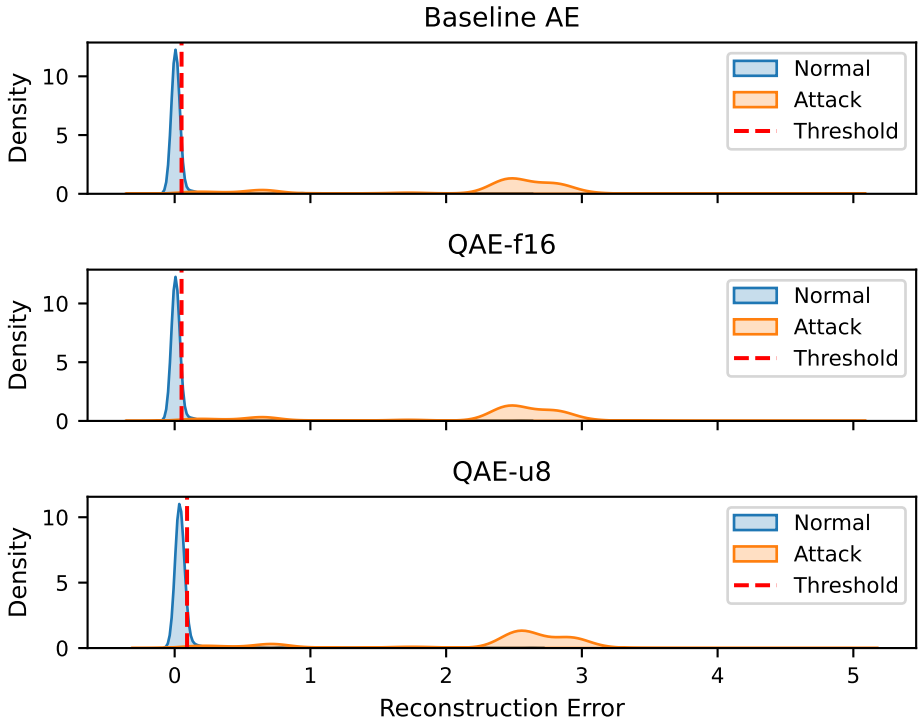

Figure 5.3: Reconstruction error distributions for normal and attack traffic across the baseline, QAE-f16, and QAE-u8 models. All models preserve strong separation between normal and attack traffic with insignificant shifts in error magnitude and threshold.

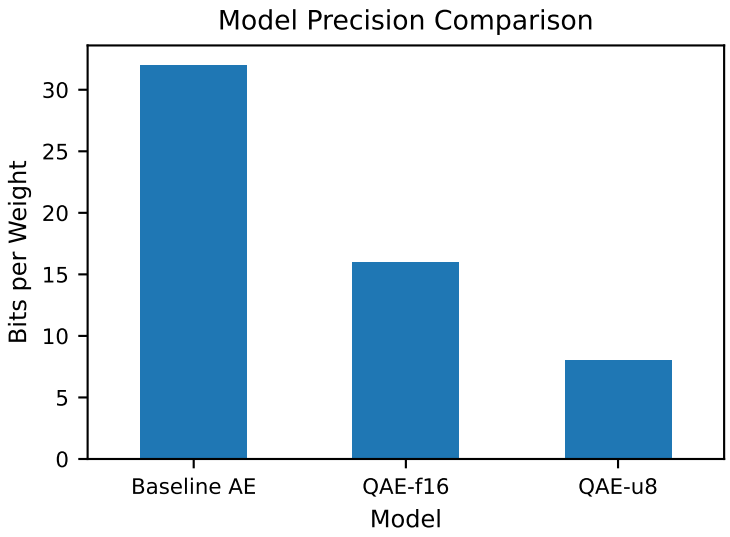

Figure 5.4: Relative parameter precision of baseline and quantized autoencoder models.

PR-AUC of **0.999898**, and an F1 score of **0.997197**, demonstrating that a simple linear decision boundary is capable of capturing most of the discriminative structure in the feature space. Feature selection using lasso and stepwise selection maintained similarly high ROC-AUC and PR-AUC performance while using only **23 and 9 features respectively**, indicating that a **reduced feature subset can retain most predictive information while improving model simplicity**.

The autoencoder-based models also achieve competitive performance. The baseline autoencoder yields a test ROC-AUC of **0.996167** and PR-AUC of **0.999533**, slightly lower in accuracy than the logistic regression models but still highly effective for anomaly detection. The quantized autoencoder variants (**QAE-f16** and **QAE-u8**) show nearly identical ROC-AUC and PR-AUC performance to the baseline autoencoder, with minimal degradation despite reduced numerical precision. This suggests that **quantization can substantially reduce model complexity and memory requirements while preserving detection capability**. Overall, these results demonstrate that while classical supervised models such as logistic regression provide strong predictive baselines, **quantized autoencoders offer a promising alternative for anomaly detection in resource-constrained IoT environments**, achieving comparable detection performance with potentially more efficient deployment characteristics.

Table 5.6: Performance comparison of logistic regression and autoencoder-based models on the RT-IoT2022 dataset.

	model	n_features	threshold	val_f1	test_roc_auc	test_pr_auc	test_accuracy	test_precision	test_recall	test_f1
0	baseline_lr	103	0.070000	0.996806	0.999055	0.999898	0.994964	0.997527	0.996866	0.997197
1	lasso_lr	23	0.060000	0.996836	0.996772	0.999587	0.994477	0.997706	0.996143	0.996924
2	stepwise_lr	9	0.050000	0.996625	0.996560	0.999626	0.994423	0.997466	0.996324	0.996894
3	Baseline AE	103	0.050757	0.995129	0.996057	0.999491	0.991986	0.993636	0.997469	0.995549
4	QAE-f16	103	0.050762	0.995129	0.996056	0.999491	0.991986	0.993636	0.997469	0.995549
5	QAE-u8	103	0.091342	0.994771	0.995955	0.999475	0.992203	0.993519	0.997830	0.995670

6 Conclusion

This study explored multiple approaches for detecting anomalous network traffic within the RT-IoT 2022 dataset, beginning with exploratory data analysis (EDA) and progressing through supervised and unsupervised modeling techniques. The objective was to evaluate how different modeling paradigms perform on IoT network telemetry and to assess the feasibility of lightweight models for deployment in resource-constrained environments.

During the **EDA phase**, we examined the distribution and structure of the dataset to better understand the underlying traffic patterns. Several key observations emerged. First, the dataset exhibited a strong imbalance between attack and normal traffic, with attack flows dominating the observations. Second, many network flow features were highly correlated, reflecting the fact that multiple telemetry metrics capture similar characteristics such as packet counts, byte volumes, and timing behavior. Finally, visualizations such as PCA projections suggested that attack traffic often occupies regions of the feature space that are distinct from benign traffic, indicating that the anomaly detection task may benefit from strong separability within the data.

We then evaluated **logistic regression models** as a supervised baseline for intrusion detection. Three variants were considered: a model trained on all features, a model using LASSO-based feature selection, and a model using stepwise selection. Logistic regression provided a simple, interpretable baseline that performed strongly despite the high dimensionality of the dataset. Feature selection helped identify a subset of predictors that retained most of the predictive signal while reducing model complexity. These results demonstrated that even relatively simple linear models can achieve strong detection performance when the underlying feature space contains clear separation between normal and attack traffic.

Next, we investigated **autoencoder-based anomaly detection**, which represents an unsupervised alternative that does not rely on labeled attack examples. The baseline autoencoder was trained exclusively on benign traffic to learn a compressed representation of normal network behavior. Reconstruction error was then used as an anomaly score, and a threshold selected using validation F1 maximization converted the continuous scores into binary predictions. The autoencoder achieved strong detection performance on the test set, indicating that reconstruction error effectively captures deviations from normal traffic patterns.

To explore deployment-friendly models, we further evaluated **quantized autoencoder variants**, including a half-precision model (QAE-f16) and an 8-bit dynamically quantized model (QAE-u8). Despite significant reductions in numerical precision, both quantized models exhibited performance nearly identical to the full-precision baseline. Although the INT8 model produced a shifted reconstruction error scale that required a different anomaly threshold, the overall classification metrics remained stable. This result suggests that reduced-precision autoencoders can preserve detection capability while substantially reducing model memory requirements, making them promising candidates for IoT and edge deployments.

Overall, the study demonstrates that both supervised and unsupervised approaches can perform effectively on the RT-IoT dataset. Logistic regression offers a lightweight and interpretable baseline, while autoencoders provide a flexible anomaly detection framework that can generalize to unseen attack patterns. Furthermore, quantization techniques show that neural network models can be compressed significantly without sacrificing performance.

6.1 Limitations

Despite the strong results observed in this study, several limitations should be acknowledged.

First, the **Amazon Alexa traffic category was missing from the available RT-IoT dataset**, even though it is documented in the original dataset description. Because this category represents normal IoT traffic, its absence reduces the diversity of benign device behaviors in the training data. As a result, the learned representations of normal traffic may not fully capture the complete range of legitimate IoT communication patterns.

Second, the dataset contains **substantial multicollinearity among many network flow features**. While neural networks can tolerate correlated inputs better than linear models, highly redundant features may reduce the effective dimensionality of the dataset and allow models to achieve strong performance by exploiting correlated patterns rather than learning deeper behavioral structures.

Third, the dataset exhibits **strong separation between normal and attack traffic**, which simplifies the classification problem relative to real-world network environments. In operational settings, attackers may intentionally mimic legitimate traffic patterns, creating significant overlap between benign and malicious feature distributions. Consequently, models trained and evaluated on this dataset may achieve higher performance than would be expected in more challenging real-world deployments.

Finally, all experiments were conducted in an **offline evaluation setting**, meaning that models were tested on pre-collected data rather than within a live network environment. Real-world intrusion detection systems must contend with additional challenges such as evolving attack strategies and heterogeneous device behaviors.

6.2 Future Work

If additional time or resources were available, several directions could further strengthen this research. One important extension would be to incorporate **missing traffic classes such as Amazon Alexa** and expand the dataset to include a wider variety of IoT devices and communication patterns. Collecting or integrating additional IoT traffic from diverse devices and deployment environments would improve the realism and robustness of the evaluation.

Future work could also explore **real-world IoT deployments**, where models are evaluated directly on edge hardware such as Raspberry Pi or embedded gateways. Measuring model size, inference latency, CPU utilization, and memory usage would provide valuable insight into the practical feasibility of deploying intrusion detection models on resource-constrained devices. Additional research could investigate **more advanced anomaly detection architectures**, including variational autoencoders, recurrent neural networks for temporal traffic patterns, or graph-based approaches that capture relationships between devices within an IoT network.

Finally, combining supervised and unsupervised techniques into **hybrid detection frameworks** may further improve detection performance. For example, autoencoders could serve as anomaly detectors that flag suspicious traffic, while supervised classifiers refine the classification of detected anomalies. Together, these extensions would help move from offline evaluation toward practical, deployable intrusion detection systems capable of protecting diverse IoT environments.

6.3 References

1. Almadhor, A., Alsubai, S., Kryvinska, N., Al Hejaili, A., Ayari, M., Bouallegue, B., & Abbas, S. (2025). Evaluating large transformer models for anomaly detection of resource-constrained IoT devices for intrusion detection system. Scientific Reports, 15, 37972. https://www.semanticscholar.org/paper/Evaluating-large-transformer-models-for-anomaly-of-Almadhor-Alsubai/ac425cacf7c3d9e729fe7dd8d2242091f24609e4
2. Chythanya, K. R., Tuteja, G., Gupta, S., Govindrao, P. S., Mahajan, S., & Singh, A. R. (2025). Efficient anomaly detection in IoT networks using logistic regression and SMOTE. Proceedings of the 6th International Conference for Emerging Technology (INCET). https://www.semanticscholar.org/paper/Efficient-Anomaly-Detection-in-IoT-Networks-Using-Chythanya-Tuteja/f46e4359ac9708b658269baeda3bb7207779adb5
3. Cohen, J. (1988). Statistical power analysis for the behavioral sciences (2nd ed.). Lawrence Erlbaum Associates. https://www.frontiersin.org/journals/psychology/articles/10.3389/fpsyg.2013.00863/full
4. Hastie, T., Tibshirani, R., & Friedman, J. (2009). The elements of statistical learning: Data mining, inference, and prediction (2nd ed.). Springer. https://link.springer.com/book/10.1007/978-0-387-84858-7
5. IBM Security. (2025). Cost of a data breach report 2025: The AI oversight gap. IBM Corporation. https://www.ibm.com/reports/data-breach
6. Patel, N. D., Rao, V. S., & Singh, A. (2024). QDNN-IDS: Quantized deep neural network based computational strategy for intrusion detection in IoT. IEEE Silchar Subsection Conference (SILCON). https://www.semanticscholar.org/paper/QDNN-IDS%3A-Quantized-Deep-Neural-Network-based-for-Patel-Rao/cb6e0039bf8bb4c92b3e48304695362f7fae7e44
7. Peng, H., Long, F., & Ding, C. (2005). Feature selection based on mutual information: Criteria of max-dependency, max-relevance, and min-redundancy. IEEE Transactions on Pattern Analysis and Machine Intelligence, 27(8), 1226–1238. https://ieeexplore.ieee.org/document/1453511
8. Putrada, A. G., & Ilhami, D. A. S. (2024). G-mean for optimum threshold in anomaly detection with autoencoder: Cyber security on the RT-IoT2022 dataset. IEEE 22nd Student Conference on Research and Development (SCOReD). https://www.semanticscholar.org/paper/G-Mean-for-Optimum-Threshold-in-Anomaly-Detection-Putrada-Ilhami/c0e1552ff8bacfba87ecbd65bc16bc80405aa68f
9. Russell, S. J., & Norvig, P. (2021). Artificial intelligence: A modern approach (4th ed.). Pearson. https://api.pageplace.de/preview/DT0400.9781292401171_A41586057/preview-9781292401171_A41586057.pdf
10. Sharmila, B. S., & Nagapadma, R. (2023). Quantized autoencoder intrusion detection system for anomaly detection in resource-constrained IoT devices using RT-IoT2022 dataset. Cybersecurity, 6(41). https://www.semanticscholar.org/paper/Quantized-autoencoder-(QAE)-intrusion-detection-for-Sharmila-Nagapadma/753f6ede01b4acaa325e302c38f1e0c1ade74f5b
11. Tejaswini, R., Abinaya, P., Anuprabha, S. S., Chidambaram, S. K., Arya, S., Choukiker, Y. K., & Bhowmick, A. (2025). Anomaly detection in IoT networks: A deep learning approach using autoencoders. IEEE Access, 13. https://ieeexplore.ieee.org/document/11077140

6.4 Appendix

1. Github Repository: https://github.com/AnhJoe/Real-Time_IoT_2022_Quantized_Auto_Encoder